UNIVERSIDAD COMPLUTENSE MADRID

ntic master

TRABAJO FINAL DE MÁSTER

Implementación de un sistema NL2SQL con enfoque RAG Multiagente

BIAI
Business Intelligence & AI

Grupo 4 – Año 2025

Alumnos:

Alejandro González Baonza

Gastón Ezequiel Rimbano Rodríguez

Jesús Abajo Magro

Manuel Fernández de Mendieta

Pablo Morales Ambrós

Sergio García Ruíz

1. Introducción..... 5
1.1. Contexto y motivación del proyecto..... 5
1.2. Objetivos del TFM..... 5
Objetivo general..... 5
Objetivos específicos..... 5
2. Problema empresarial y aplicabilidad..... 6
2.1.Causas de problemas..... 6
2.2. Consecuencias para el negocio..... 6
2.3. Aplicabilidad multisector..... 7
2.4. Ventaja frente a alternativas y beneficios clave..... 7
3. Marco Teórico..... 8
3.1. Conceptos básicos de sistemas multiagente..... 8
3.2. Arquitectura RAG (Retrieval-Augmented Generation)..... 8
3.3. Procesamiento de lenguaje natural aplicado a bases de datos..... 9
3.4. Estado del arte: soluciones existentes en el mercado..... 9
4. Metodología y Desarrollo del Proyecto..... 11
4.1. Enfoque metodológico general..... 11
4.2. Tecnologías utilizadas..... 12
4.3. Arquitectura general del sistema..... 12
4.4. Componentes principales..... 13
4.4.1. Agente orquestador..... 14
4.4.2. Agente de Generador de Contexto RAG..... 14
4.4.3. Agente de generación de queries..... 14
4.4.4. Agente de validación de queries..... 14
4.5.5. Agente generador de conclusiones..... 15
4.4.6. RAG: Base de datos de conocimiento..... 15
4.4.7. Interfaz..... 16
5. Aplicación práctica..... 16
5.1. Caso de uso: AdventureWorks como empresa de referencia..... 16
5.2. Acceso al repositorio del proyecto..... 17
6. Evaluación del Sistema..... 17
6.1. Metodología de pruebas..... 17
6.2. Pruebas sobre LLM's..... 17
7. Escalado..... 18
7.1. Beneficios y diferenciación frente a alternativas..... 18
7.2. Posibilidades de escalado..... 18
8. Conclusiones y Trabajo Futuro..... 19
8.1. Resumen de logros del proyecto..... 19
8.2. Limitaciones actuales..... 19
8.3. Posibles mejoras y extensiones..... 19
8.4. Perspectiva de implantación real en empresas..... 20
9. Bibliografía..... 21
10. Anexos..... 22

10.1. Resumen de tecnologías utilizadas....22
10.2. Modelos de Lenguaje Utilizados (LLM's)....24
10.3 Retrieval Augmented Generation (R.A.G.)....26
10.3.1. Colecciones - Configuración....26
10.3.1.1. Función de Distancia....26
10.3.1.2. Modelos de Embeddings....27
10.3.2. Colecciones - Áreas de Conocimiento....28
10.3.2.1. Esquema de Datos - MDL....29
10.3.2.1.1. Chunking....29
mdl_tables_summary....30
mdl_columns....30
10.3.2.1.2. Retrieval....31
10.3.2.2. Lógica de Negocio....32
10.3.2.2.1. Chunking....32
10.3.2.2.2. Retrieval....33
Método de Búsqueda - Maximal Marginal Relevance (MMR)....33
Técnica de Recuperación - Context Enrichment Window....33
10.3.2.3. Ejemplos de Consultas SQL....34
10.3.2.3.1. Chunking....34
10.3.2.3.2. Retrieval....35
10.4. Explicación Detallada de la Arquitectura del Sistema....36
10.4.1. Agente orquestador....37
10.4.1.1. detect_user_query_language....38
10.4.1.2. grade_question_relevance....38
10.4.1.3. structure_final_output....39
10.4.2. Agente generador de contexto....39
10.4.2.1. get_business_logic & get_data_schema....40
10.4.2.1.1. create_sub_queries....41
10.4.2.1.2. retrieve_chunks....41
10.4.2.1.3. process_chunks....41
10.4.2.1.4. create_summary....42
10.4.2.2. extract_db_info....42
10.4.2.3. grade_context_summaries....42
10.4.2.4. get_global_context....43
10.4.2.5. get_no_relevance_response....43
10.4.3. Agente generador de queries....43
10.4.3.1. query_examples_retriever....44
10.4.3.2. query_generator....44
10.4.4. Agente validador de queries....44
10.4.4.1. check_db_connection....45
10.4.4.2. tables_extraction....46
10.4.4.3. validation....46
10.4.4.4. query_correction....46
10.4.5. Agente generador de conclusiones....46

10.4.5.1. generate_nl_conclusions....47
10.4.5.2. generate_sql_explanation....47
10.4.5.3. generate_plotly_graphics....48
10.5. Gradio....48
10.5.1. Estructura de la interfaz....48
Pestañas principales....48
Secciones y Columnas....49
Barra lateral (Sidebar)....50
Componentes de visualización de datos....50
10.5.2. Ejemplos....51
10.5.2.1. Ejemplo 1....51
Consulta realizada....51
Conclusiones....51
Explicación de Query....52
Análisis de la Lógica de la Consulta....52
Conclusión y Resultados Clave....53
10.5.2.2. Ejemplo 2....53
Consulta realizada....53
Conclusiones....54
Explicación de Query....55
Analysis of the Query Logic....55
Conclusion and Key Results....55
10.5.2.3. Ejemplo 3....56
Consulta realizada....56
Conclusiones....56
Explicación de Query....57
Análisis de la Lógica de la Consulta....57
Conclusión y Resultados Clave....57
10.6. Conjunto de Datos....59

1. Introducción

1.1. Contexto y motivación del proyecto

En la mayoría de las empresas, acceder a la información no siempre es sencillo. Los equipos de negocio como marketing, operaciones o contabilidad, dependen del área técnica para obtener datos de las bases corporativas. Esta dinámica genera cuellos de botella: el personal técnico dedica gran parte de su tiempo a tareas rutinarias de extracción y validación, mientras que las áreas de negocio esperan para tomar decisiones que deberían ser rápidas.

Este proyecto busca eliminar esas barreras. La propuesta es una solución que actúa como un equipo de asistentes virtuales capaces de entender preguntas en lenguaje natural y buscar la información necesaria en las bases de datos corporativas. De este modo, cualquier empleado puede acceder a los datos que necesita sin conocimientos técnicos y recibir respuestas claras y seguras. Así se consigue:

- **Autonomía** para los departamentos de negocio en sus consultas más habituales.
- Decisiones más ágiles gracias a la **reducción de tiempos de espera**.
- Más tiempo para que los equipos técnicos se **concentren en innovación y proyectos estratégicos**.

1.2. Objetivos del TFM

Objetivo general.

Diseñar e implementar un asistente de autoservicio de datos que entienda preguntas de negocio en lenguaje natural y devuelva respuestas claras obtenidas de las bases de datos corporativas. La solución se apoya en un “equipo” de asistentes virtuales especializados (agentes) que se reparten el trabajo: entender la pregunta, localizar los datos, generar y revisar la consulta en base de datos y elaborar la respuesta.

Objetivos específicos

O1. Definir una **arquitectura modular basada en agentes** (orquestación, conocimiento, generación y validación de consultas, y redacción de respuestas) con criterios básicos de seguridad y gobierno del dato.

O2. Desarrollar el **módulo de contexto** que aporte glosario, metadatos y ejemplos del modelo de datos para guiar la traducción de lenguaje natural a SQL.

O3. Implementar un **prototipo funcional** con AdventureWorks (dataset público de ejemplo) que refleja un esquema dimensional típico de Data Warehouse sobre un motor relacional estándar.

O4. Proponer líneas de evolución: revisor automático de respuestas (LLM-as-a-judge), sistema de retroalimentación de usuarios (valoraciones y comentarios), optimización del rendimiento, soporte multilingüe y actualización dinámica del conocimiento.

2. Problema empresarial y aplicabilidad

2.1.Causas de problemas

Una serie de factores comunes suele estar en el origen de las ineficiencias y errores en entornos empresariales. A continuación, se resumen las **causas típicas** que afectan a sistemas de información, gestión de datos y flujos de procesos internos:

- **Procesos obsoletos o mal diseñados**: procedimientos sin revisar que acumulan pasos redundantes, papeleo y excepciones; elevan complejidad, riesgo de error y retrabajo.
- **Sistemas aislados y silos de información:** herramientas y bases desconectadas por área; obliga a buscar y consolidar manualmente, genera duplicidades o inconsistencias y ralentiza el flujo entre equipos.
- **Falta de automatización y herramientas adecuadas:** tareas manuales (carga de datos, informes, conciliaciones) que no escalan; mayor propensión a errores y costes de operación.
- **Comunicación deficiente y departamentos desconectados:** la falta de coordinación interdepartamental y responsabilidades difusas; aparecen duplicidades, esperas para conseguir datos y decisiones con información parcial.
- **Sobrecarga de trabajo y gestión del tiempo ineficiente:** planificación deficiente que satura equipos, crea cuellos de botella y aumenta errores en picos de demanda.

2.2. Consecuencias para el negocio

Las ineficiencias descritas tienen un impacto directo en la operación, los costes y la relación con el cliente. Los principales efectos son:

- **Baja productividad y retrabajo.** Se invierte demasiado tiempo en buscar información, corregir errores y repetir tareas. Esto reduce la capacidad para dedicarse a actividades de mayor valor y alarga los ciclos de trabajo.
- **Aumento de costes.** Más horas en tareas manuales, correcciones y coordinaciones innecesarias, además de un uso ineficiente de recursos e infraestructuras.
- **Retrasos y tiempos de respuesta lentos.** Los cuellos de botella y la falta de coordinación provocan incumplimientos de plazos, afectando a ventas, soporte y operaciones.
- **Menor calidad en el servicio y en la información.** Los errores, las definiciones poco claras y la falta de control generan informes poco fiables y decisiones equivocadas; también empeoran la experiencia del cliente y dificultan la auditoría.
- **Clientes insatisfechos y pérdida de confianza.** Las incidencias repetidas y las respuestas tardías o contradictorias generan más reclamaciones y reducen la fidelidad de los clientes.
- **Reputación y competitividad debilitadas.** La percepción de lentitud y desorden resta credibilidad, dificulta diferenciarse en el mercado e inhibe la inversión en innovación; a medio plazo, esto puede afectar tanto a la cuota de mercado como a la rentabilidad.

2.3. Aplicabilidad multisector

Estos casos muestran que la herramienta no solo resuelve un problema técnico, sino que **abre nuevas posibilidades de gestión** al poner los datos en manos de quienes toman las decisiones, sin fricciones ni retrasos.

La necesidad de acceder a la información en lenguaje natural no es exclusiva de un tipo de organización. Al contrario, se trata de un problema transversal que afecta a múltiples sectores donde el volumen de datos es elevado y las decisiones deben tomarse con rapidez. En todos estos casos, la dependencia de consultas SQL o de equipos técnicos intermedios se convierte en un freno operativo.

El sistema propuesto en este TFM elimina esa barrera al permitir que cualquier usuario autorizado formule preguntas en lenguaje natural y reciba respuestas claras y validadas. Esta capacidad resulta especialmente valiosa en sectores como los siguientes:

- **Retail omnicanal y comercio electrónico**: donde la rotación de SKUs, las devoluciones y el pricing cambian día a día. Los responsables de compras o catálogo pueden consultar, por ejemplo, el *Top-10 SKUs por rotación en tiendas físicas* frente *a ventas online en el mes actual*, y recibir un ranking acompañado de recomendaciones de reposición.
- **Logística y operadores 3PL**: donde el cumplimiento de SLA y las demoras en ruta impactan directamente en la satisfacción del cliente. Con una simple consulta como *"¿On-time delivery por cliente y ruta en la última semana?"*, los equipos de operaciones acceden a un panel inmediato con alertas de desvío que facilita la optimización de rutas.
- **Fabricación industrial**: donde indicadores como el OEE, las causas de paro de línea o el scrap afectan a la eficiencia de planta. Un jefe de producción puede preguntar *"¿Cuáles han sido las causas de paro más frecuentes en los últimos 30 días por línea?"* y obtener un Pareto de paros que permite enfocar acciones de mejora de inmediato.

2.4. Ventaja frente a alternativas y beneficios clave

Frente a los dashboards tradicionales, rígidos y dependientes de perfiles técnicos, esta propuesta ofrece una **interacción conversacional directa** con la base de datos, sin necesidad de SQL. La incorporación de contexto empresarial, como definiciones internas de KPIs o reglas de negocio, asegura que las respuestas no sean genéricas, sino coherentes con la operativa de cada organización.

Las principales ventajas se resumen en:

- **Rapidez y autonomía**: respuestas en tiempo real, eliminando colas de soporte técnico.
- **Confiabilidad**: validación automática de consultas y trazabilidad del proceso.
- **Adaptabilidad**: integración con bases de datos existentes y escalabilidad hacia nuevas fuentes.
- **Acceso inclusivo**: cualquier usuario autorizado puede obtener respuestas en su propio idioma, con presentaciones claras y, cuando aplica, visualizaciones que facilitan la interpretación.

En definitiva, la herramienta convierte la interacción con los datos en una experiencia accesible, transparente y alineada con las necesidades del negocio, aportando un valor estratégico en sectores donde la agilidad y la precisión son diferenciales.

3. Marco Teórico

3.1. Conceptos básicos de sistemas multiagente

Un **sistema multiagente (MAS)** es un conjunto de **agentes autónomos** que cooperan, coordinan o incluso compiten para alcanzar objetivos. Cada **agente** es una entidad que **percibe su entorno, toma decisiones y actúa** de forma relativamente independiente.

Características básicas:

- **Autonomía:** no requieren supervisión constante.
- **Reactividad y proactividad:** responden a cambios y persiguen metas propias.
- **Sociabilidad:** interactúan entre sí mediante comunicación.

Se parecen a un equipo de trabajo, en el que cada miembro tiene un rol específico y todos colaboran hacia una meta común.

Ventajas principales: escalabilidad (es fácil añadir agentes), robustez (si uno falla, el sistema puede seguir funcionando) y modularidad (los roles se adaptan).

Limitaciones: coordinación compleja, riesgo de conflictos entre objetivos y mayor dificultad para garantizar comportamientos previsibles.

3.2. Arquitectura RAG (Retrieval-Augmented Generation)

Cuando alguien pregunta en lenguaje natural, el sistema necesita contexto para responder con precisión. La arquitectura **RAG** hace dos cosas: **(1) busca** fragmentos de información relacionados con la pregunta y **(2) genera** una respuesta usando los mismos como contexto. Para lograrlo, se trabaja con una base de datos vectorial que utiliza un **índice de embeddings** para generar una representación semántica de los contenidos que se almacenan.

¿Por qué es clave en este TFM?

- **Traduce** términos cotidianos a los nombres reales del modelo de datos (tablas/columnas).
- **Enriquece** la consulta con conceptos cercanos (p. ej., “ingresos” → *TotalDue*, *SalesAmount*).
- **Reduce errores y ambigüedades**, al trabajar con fragmentos relevantes.
- **Escala** a distintas bases de datos sin que el modelo “memorice” cada detalle.

Ejemplo breve.

Si un usuario consulta por la rentabilidad del trimestre pasado, aunque en los documentos de la base vectorial no exista esa definición como tal, el sistema es capaz de capturar conceptos relevantes para construir una consulta SQL adecuada, sin que tenga que intervenir ni expertos de negocio, ni IT.

Explicación técnica ampliada y funcionamiento interno: **véase Anexo 10.3**.

3.3. Procesamiento de lenguaje natural aplicado a bases de datos

El **Procesamiento de Lenguaje Natural** (PLN o NLP, por sus siglas en inglés) es la rama de la inteligencia artificial que permite que las máquinas comprendan, interpreten y generen lenguaje humano. En este proyecto, su papel es fundamental: es lo que hace posible que un usuario formule una consulta como lo haría en una conversación con un analista de negocio, por ejemplo: "¿Cuántos pedidos se realizaron el último mes en la zona norte?" y que el sistema la transforme automáticamente en una instrucción SQL válida.

Este enfoque, conocido como **NLIDB (Natural Language Interface to Databases)**, elimina la necesidad de que los usuarios adapten su forma de expresarse o tengan que recurrir a tecnicismos. En lugar de aprender SQL o conocer el esquema de la base de datos, basta con que planteen su pregunta en lenguaje cotidiano: el sistema se encarga de interpretar el significado y de generar la consulta correspondiente.

Los principales retos de este proceso son:

- **Ambigüedad del lenguaje humano**, donde un mismo término puede tener múltiples interpretaciones (por ejemplo, "clientes activos").
- **Diferencia entre lenguaje cotidiano y esquemas técnicos**, ya que conceptos como "ventas" o "ingresos" se registran en columnas con nombres distintos en la base de datos.
- **Corrección de errores en consultas**, para evitar que se generen sentencias incompletas o inválidas.
- **Presentación de resultados**, que deben mostrarse de forma clara y contextualizada, más allá de simples tablas en bruto.

En el marco de este TFM, el PLN no actúa de manera aislada, sino integrado con la arquitectura **RAG** y la validación iterativa del sistema multiagente. Gracias a esta combinación, la interacción con la herramienta es natural y accesible: el usuario puede comunicarse como lo haría con un analista de BI, y recibir respuestas precisas, explicadas y listas para la toma de decisiones.

3.4. Estado del arte: soluciones existentes en el mercado

En los últimos años se ha incrementado el interés por herramientas que permiten preguntar a las bases de datos en lenguaje natural. Algunos artículos reflejan este auge: por ejemplo, desde la comunidad técnica se explica cómo han ido evolucionando estas arquitecturas y los retos que afrontan para mejorar la precisión y la escalabilidad *(Medium, 2024)* *[1]*. A su vez, Microsoft ha planteado la creación de agentes de datos inteligentes con **Semantic**

Kernel, capaces de transformar preguntas en consultas de manera segura y alineada con el entorno empresarial *(Microsoft Tech Community, 2024)* [2].

Estas propuestas muestran que existe un ecosistema en crecimiento, donde distintas soluciones aportan ventajas pero también limitaciones. Nuestro proyecto busca precisamente unificar lo mejor de estos enfoques en una sola herramienta que combine simplicidad, capacidad técnica y adaptación a las necesidades de negocio.

Además de las mejoras frente a los métodos tradicionales ya mencionadas, resulta necesario revisar qué alternativas se han desarrollado con un enfoque similar al nuestro. En el mercado existen diferentes soluciones, como **Chat2BD** [3], **Uber QueryGPT** [4] o **Alation** [5], que permiten formular preguntas en lenguaje natural y obtener información a partir de las bases de datos.

Cada una de estas plataformas aporta funcionalidades valiosas, pero también presenta limitaciones: unas destacan en usabilidad, otras en precisión o en capacidades de integración, pero ninguna reúne todas esas ventajas en una única solución.

Con nuestro proyecto hemos buscado precisamente unificar lo mejor de cada enfoque, de modo que la herramienta ofrezca un conjunto completo de funcionalidades, combinando simplicidad de uso, potencia técnica y flexibilidad para distintos perfiles de usuario.

En la **figura 3.4** se pueden observar las características de cada uno de las soluciones

	Chat2DB	Uber QueryGPT	Alation	BiAi
Integración con datos empresariales	✓	✓	✓	✓
Generación NL→SQL	✓	✗	✗	✓
Validación y corrección de queries	✗	✗	✓	✓
Explicación de la lógica de la consulta	✓	✗	✗	✓
Presentación de resultados	✗	✓	✓	✓

Figura 3.4 Tabla comparativa

De este modo, el sistema no sustituye completamente a las herramientas existentes, sino que cubre el vacío entre la flexibilidad de SQL, la rigidez de los dashboards y la falta de contexto de los LLM genéricos, posicionándose como una solución intermedia y diferenciada.

4. Metodología y Desarrollo del Proyecto

4.1. Enfoque metodológico general

El trabajo se ha desarrollado siguiendo un enfoque de investigación aplicada, orientado a la construcción de un prototipo funcional que permitiera validar la viabilidad de un sistema multiagente basado en RAG para la consulta en lenguaje natural de bases de datos corporativas.

Más que plantearse como un desarrollo puramente teórico, el proyecto ha buscado desde el inicio combinar dos dimensiones complementarias:

- Por un lado, el rigor académico, mediante la revisión bibliográfica y el análisis del estado del arte en sistemas multiagente, RAG y procesamiento de lenguaje natural.
- Por otro lado, la orientación práctica, a través del diseño e implementación de un sistema que pueda aplicarse en un escenario empresarial real.

El trabajo se ha organizado en varias fases iterativas:

1. **Revisión del estado del arte y definición del problema:** se investigó la situación actual, analizando qué soluciones habían sido desarrolladas, qué planteaban las fuentes especializadas y se identificaron las principales barreras que aún encuentran los usuarios de negocio en el día a día.
2. **Diseño conceptual de la arquitectura:** se estableció el modelo a seguir, se seleccionaron las tecnologías más adecuadas y se definió el alcance del sistema.
3. **Implementación modular de agentes**: el equipo de trabajo dividió la carga de desarrollo, de modo que cada integrante fue responsable de diseñar y probar un agente concreto. Este enfoque permitió asegurar que cada componente cumpliera con su función de manera aislada antes de integrarse en el sistema completo.
4. **Pruebas individuales y globales**: primero se realizaron pruebas unitarias sobre cada agente de manera independiente, verificando su correcto funcionamiento; posteriormente, se integraron todos bajo el agente orquestador, lo que permitió comprobar la coherencia del flujo global de información.
5. **Evaluación en un entorno de pruebas**: una vez validado el sistema completo, se utilizó una base de datos representativa de un escenario empresarial para simular consultas reales y analizar el valor añadido que aporta el enfoque.
6. **Despliegue en contenedor:** como último paso, la herramienta se preparó con Docker para garantizar que el proyecto fuera exportable y pudiera ejecutarse sin dificultades en distintos entornos.

Esta metodología permitió avanzar de forma iterativa y controlada, validando en cada fase que el sistema cumplía con los objetivos planteados y que mantenía la coherencia entre la visión técnica y el impacto empresarial buscado.

4.2. Tecnologías utilizadas

El sistema se ha desarrollado con **Python** (prototipado en notebooks y consolidación en módulos), **Docker** (portabilidad y despliegue reproducible), **PostgreSQL** (SGBD de referencia en servicio gestionado), **LangGraph** (orquestación de agentes como grafo), ChromaDB (almacenamiento vectorial para recuperación semántica), **Vertex AI** de Google (embeddings multilingües para RAG) y **Azure AI Foundry** con modelos de OpenAI (interpretación de lenguaje natural).

La configuración y ejemplos se detallan en el **Anexo 10.1**

4.3. Arquitectura general del sistema

La arquitectura del sistema desarrollado se concibe como un pipeline de procesamiento en varias capas, en el que la información fluye desde la consulta del usuario hasta la respuesta final validada y presentada en distintos formatos.

En la **figura 4.3** se ha propuesto un esquema de la arquitectura del sistema

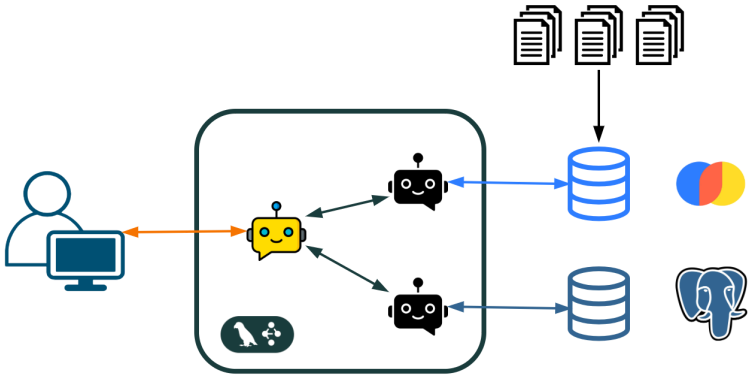


Figura 4.3 Esquema de arquitectura

Interacción del usuario: en primer lugar, el usuario accede a una interfaz gráfica construida con **Gradio**. Desde este entorno puede plantear una pregunta y seleccionar el número de líneas que desea en la respuesta.

Resultados ofrecidos: tras un breve tiempo de procesamiento, el sistema devuelve tres salidas principales:

- Una **explicación en texto**, adaptada al lenguaje del usuario.
- Un **archivo Excel/CSV** con los datos estructurados, que puede descargarse y utilizarse en otras herramientas.
- Una **visualización gráfica**, similar a la que ofrecen plataformas de business intelligence como Power BI, para facilitar la interpretación de la información.

Además, la interfaz muestra la **consulta SQL generada y ejecutada** sobre la base de datos, lo que añade transparencia al proceso.

Procesamiento interno: una vez lanzada la consulta, el flujo queda en manos de la **arquitectura multiagente** coordinada por un orquestador general. Este asegura que cada agente intervenga en el momento adecuado, compartiendo contexto mediante **LangGraph**.

Dentro de este pipeline:

- El **agente RAG** recupera fragmentos relevantes desde la base vectorial, construida a partir del esquema de datos.
- El **agente de validación** ejecuta la sentencia SQL sobre la base de datos relacional (AdventureWorks, en este prototipo) para comprobar su validez y extraer resultados.
- El **agente generador de conclusiones** organiza y presenta la información, generando una explicación textual y una estructura en formato **JSON**, que luego es utilizada por la capa de interfaz para maquetar la respuesta.

  Una descripción detallada de la arquitectura se encuentra en el **Anexo 10.4**

4.4. Componentes principales

Consta de los siguientes elementos:

- El **agente orquestador.**
- El **agente de generación de contexto RAG**.
- El **agente generador de queries.**
- El **agente corrector de queries.**
- Finalmente, el **agente generador de conclusiones**.

La ventaja de este enfoque es que cada "miembro" (agente) **está especializado en su tarea**, y al trabajar en conjunto logran resolver un problema complejo de forma más eficiente que si un único agente intentará hacerlo todo. Además, este diseño modular permite que, en el futuro, se puedan añadir o sustituir agentes sin necesidad de rediseñar todo el sistema, lo que aporta **flexibilidad y escalabilidad**.

En la **figura 4.4** se muestra un esquema del funcionamiento de los agentes

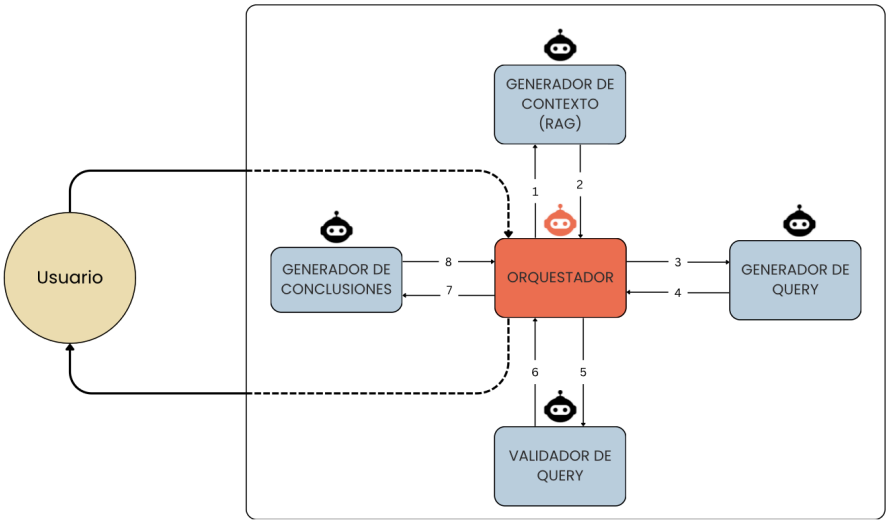


Figura 4.4 Esquema de los agentes

4.4.1. Agente orquestador

Actúa como jefe de proyecto. Organiza el flujo de tareas y decide a quién asignar cada paso. Conectado a Azure AI Foundry, consume modelos de OpenAI de forma compatible con el marco regulatorio europeo (GDPR). Interpreta la consulta del usuario y determina si está vinculada con el data warehouse; en caso contrario, responde de manera cordial y redirige la conversación hacia temas relevantes. Además, detecta el idioma de la consulta para asegurar que toda la interacción,incluida la respuesta final, se mantenga en el idioma del usuario.

El esquema detallado del funcionamiento de este agente se incluye en el **Anexo 10.4.1**

4.4.2. Agente de Generador de Contexto RAG

Sería como un **analista** que recopila la información necesaria para entender la situación antes de tomar una decisión y actúa como puente **de información**. Recupera el contexto relevante del esquema de la base de datos y lo combina con la pregunta del usuario. Toda esa información enriquecida se envía como entrada a los agentes posteriores. Gracias a este paso, el generador y el validador de consultas disponen de la información clave que necesitan para elaborar queries más precisas y coherentes. Si el RAG detecta que la información solicitada no está en las tablas disponibles, lo comunicará, evitando respuestas erróneas. En lugar de responder de manera negativa, enviará información sobre qué tipo de consultas es capaz de responder con la información almacenada.

La explicación detallada de este agente se encuentra en el **Anexo 10.4.2**

4.4.3. Agente de generación de queries

Este agente se encarga de traducir la petición del usuario a una consulta SQL inicial. Para ello, combina el contexto proporcionado por el agente de recuperación previa (RAG) con ejemplos almacenados en una base vectorial de queries, lo que le permite inspirarse en patrones similares ya resueltos. Su objetivo es formular una sentencia SQL que represente con fidelidad la intención expresada en lenguaje natural.

El desglose del funcionamiento de este agente puede consultarse en el **Anexo 10.4.3**

4.4.4. Agente de validación de queries

El agente validador entra en juego tras la generación de la consulta inicial y asume un papel de control de calidad. Su cometido se centra en dos aspectos fundamentales:

- **Coherencia semántica**: verificar que la sentencia SQL responda de manera adecuada a la intención expresada en la pregunta del usuario.
- **Correctitud técnica o sintaxis**: comprobar que la consulta sea sintácticamente válida y pueda ejecutarse correctamente sobre la base de datos.

Si se detecta algún fallo, ya sea de sentido o de ejecución, la consulta se devuelve al agente generador junto con la retroalimentación necesaria para ajustarla. Este ciclo de generación y validación puede repetirse hasta un máximo de cinco iteraciones, asegurando que la query final sea consistente, ejecutable y alineada con la pregunta del usuario antes de continuar en el pipeline.

El detalle ampliado sobre este agente figura en el **Anexo 10.4.4**

4.5.5. Agente generador de conclusiones

Este agente actúa como el comunicador del sistema, encargado de transformar los resultados técnicos en una salida clara y comprensible para el usuario final. Tras validar la consulta y obtener los datos, el agente cumple tres funciones principales:

- **Interpretación de los resultados**: analiza la información devuelta por la base de datos y extrae conclusiones relevantes, evitando limitarse a mostrar una tabla en bruto.
- **Evaluación de la consulta**: complementa el análisis ofreciendo también una valoración de la propia query, lo que permite explicar posibles limitaciones o matices en la forma en que se construyó la respuesta.
- **Generación de visualizaciones**: convierte los resultados en gráficos o representaciones visuales que facilitan la comprensión de tendencias, comparaciones o relaciones entre datos.

De esta manera, el agente no solo traduce el lenguaje SQL en información natural, sino que además enriquece la interacción con conclusiones interpretadas y representaciones gráficas, cerrando el pipeline de extremo a extremo.

La documentación técnica de este agente se recoge en el **Anexo 10.4.5**

4.4.6. RAG: Base de datos de conocimiento

La base de conocimiento que es consultada por el RAG del sistema está especialmente diseñada para alcanzar el objetivo perseguido por el mismo. La misma se compone de 3 áreas de conocimiento principalmente, que nos permite acometer los diferentes equipos implicados:

- **Esquema de datos**: Mediante un documento principalmente técnico se logra capturar la estructura de los datos, ampliando a la vez el contexto de los mismos al añadir descripciones significativas.
- **Lógica de negocio**: Este conocimiento se captura de un documento markdown escrito en un lenguaje simple y descriptivo, como podría ser explicado por un usuario de negocio durante una reunión con el equipo técnico.
- **Ejemplos de consultas**: Estas consultas ya validadas por un equipo técnico aseguran que el agente generador de consultas disponga de ejemplos relevantes para la pregunta del usuario.

Esta combinación asegura que el agente generador de contexto pueda nutrirse tanto del enfoque técnico como del de negocio, asegurando que el agente generador de consultas disponga de la información necesaria para interpretar las preguntas del usuario, y así generar consultas SQL más precisas. Es importante comentar que desde un punto de vista técnico, para cada una de estas áreas de conocimiento, se ha desarrollado un enfoque de procesamiento de la información que maximiza la relevancia del contexto obtenido al consultar la misma, lo que permite ofrecer respuestas más claras y relevantes para cualquier perfil dentro de la organización.

La información técnica sobre este sistema se encuentra disponible en el **Anexo 10.3.2**

4.4.7. Interfaz

La interfaz es el único punto de contacto entre el usuario y el sistema, por lo que se ha diseñado para ser simple e intuitiva. El objetivo es que cualquier persona pueda usarla sin necesidad de conocimientos técnicos, comprendiendo fácilmente la información que recibe.

Para adaptarse a distintas necesidades, se han implementado diferentes vistas: una visión en resumen para captar la idea principal, una en dato para ver el valor exacto, una en gráfico para interpretar tendencias de forma visual y la propia consulta SQL para quienes deseen revisar el detalle técnico.

De este modo, la interfaz, mostrada en la **figura 4.4.7** ofrece un abanico completo de posibilidades, combinando claridad y flexibilidad en un mismo entorno.

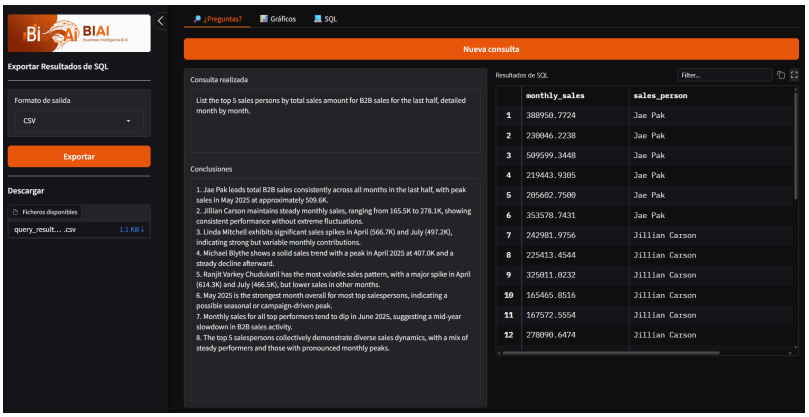


Figura 4.4.7 Interfaz de usuario

La descripción técnica de la interfaz y de cada vista se recoge en el **Anexo 10.5**

5. Aplicación práctica

5.1. Caso de uso: AdventureWorks como empresa de referencia

Para ejemplificar el problema se ha utilizado **AdventureWorks Cycles**, una empresa ficticia de ventas de bicicletas y accesorios. Su base de datos corporativa recoge clientes, pedidos, productos y transacciones, convirtiéndose en la fuente principal para áreas como marketing, y relevante para áreas como operaciones o contabilidad.

En el **modelo de trabajo tradicional**, un empleado de negocio plantea su necesidad a IT: desde preguntas simples (“¿cuántas bicicletas Mountain-200 se vendieron el último trimestre?”) hasta comparativas complejas entre regiones o períodos. El equipo técnico debe revisar el esquema, redactar la consulta, verificar resultados y atender posibles cambios.

Un ejemplo claro es el de **marketing**: requiere datos frecuentes para segmentar clientes y evaluar campañas, pero al no manejar SQL debe recurrir siempre a IT. El resultado son retrasos en la planificación y una carga rutinaria para el área técnica.

Con la implantación del sistema propuesto, el flujo cambia. El propio departamento de marketing puede escribir directamente: “Muéstrame los cinco productos más vendidos en

Europa el mes pasado". El sistema interpreta la petición, localiza los campos adecuados, construye y valida la consulta y devuelve un resultado explicado. De este modo, marketing gana autonomía e IT dedica más tiempo a innovación y proyectos de valor.

El efecto se extiende a otros departamentos de AdventureWorks: contabilidad consulta ingresos del último trimestre,dirección evalúa la evolución de ventas por región. En todos los casos, se consigue **mayor velocidad en la toma de decisiones**, **menos dependencia de IT** y **mejor aprovechamiento del talento técnico**.

> La descripción detallada de la base de datos AdventureWorks, incluyendo el modelo de datos, tipos de tablas y dimensiones, así como ejemplos adicionales de consultas, se recoge en el **Anexo 10.6**.

5.2. Acceso al repositorio del proyecto

Con el objetivo de **difundir el trabajo desarrollado** y dejar constancia de su implementación, se ha habilitado un repositorio en GitHub con el código fuente y documentación asociada. Este recurso funciona como punto de referencia complementario a la memoria, facilitando que el proyecto quede accesible de manera abierta y pueda servir de base para futuras consultas o desarrollos relacionados.

El repositorio está disponible en la siguiente dirección:
https://github.com/grimbano/ucm-tfm-grupo-4

6. Evaluación del Sistema

6.1. Metodología de pruebas

La evaluación se planteó de forma progresiva para minimizar riesgos y asegurar calidad en cada paso. En una primera fase se validó **cada agente de manera aislada**, comprobando que su función (recuperación de contexto, generación de SQL, validación y redacción) se ejecutaba con coherencia y sin errores. Esta verificación "pieza a pieza" permitió depurar mensajes, prompts y reglas antes de integrarlos.

En una segunda fase se abordó la **prueba end-to-end** bajo el orquestador: desde la pregunta en lenguaje natural hasta la respuesta explicada (texto + SQL + resultado estructurado). Se observó un flujo estable, sin bloqueos entre etapas, y una trazabilidad suficiente para comprender cómo se llegó a cada respuesta. Este enfoque escalonado, primero unitario y después integrado, confirma la **viabilidad técnica** del sistema y sienta la base para su traslado controlado a entornos reales.

6.2. Pruebas sobre LLM's

Evaluación de modelos: en lugar de realizar pruebas específicas sobre escenarios externos, la valoración se centró en determinar qué modelos de lenguaje (LLM) ofrecían un mejor desempeño para cada tipo de tarea dentro del sistema. Este análisis permitió asignar de manera óptima los modelos según sus características de coste, calidad y latencia, garantizando un equilibrio adecuado entre eficiencia y resultados.

La información detallada sobre esta medición, junto con las comparativas entre modelos, se encuentra en el **Anexo 10.2**

7. Escalado

7.1. Beneficios y diferenciación frente a alternativas

La evaluación realizada muestra que las tecnologías tradicionales para el acceso a datos, como las consultas SQL manuales o las plataformas de BI clásicas, tienden a ser sustituidas o, al menos, complementadas por nuevas herramientas que operan con agentes inteligentes. Este cambio responde a la necesidad de mayor flexibilidad, agilidad y accesibilidad en el uso de la información dentro de las organizaciones.

En este contexto, el sistema desarrollado ha demostrado que es posible unificar en una sola herramienta funcionalidades que hasta ahora aparecían de forma dispersa: consultas en lenguaje natural, validación automática, transparencia en la lógica seguida, exportación en distintos formatos y visualización gráfica. Estos beneficios no son solo un objetivo teórico, sino resultados ya conseguidos en el prototipo construido durante el proyecto.

Con ello, se confirma que la solución cumple con lo definido en el alcance, ofreciendo una herramienta completa, viable técnicamente y con valor práctico en escenarios empresariales reales.

7.2. Posibilidades de escalado

El potencial de escalado del sistema no se limita a los departamentos más evidentes como marketing o contabilidad, sino que abre oportunidades en áreas que habitualmente quedan fuera del acceso directo a bases de datos por el elevado coste que supondría dotarlas de soporte técnico, como:

- **Recursos Humanos**: consulta de indicadores de rotación, absentismo y evolución de nóminas sin depender de informes puntuales de IT.
- **Gestión de Proyectos**: seguimiento en tiempo real del estado de entregables y presupuestos, con capacidad de análisis inmediato.
- **Compras**: identificación de proveedores con mayores retrasos y análisis de la evolución histórica de precios para apoyar decisiones de negociación.

Que estos usos no sean habituales en sistemas tradicionales se explica por el **coste** de dedicar **personal técnico** a consultas ad-hoc en dichos departamentos. Con este sistema, ese cuello de botella desaparece: el acceso se estandariza y el retorno de la inversión mejora.

La escalabilidad no se limita al número de usuarios o áreas; tambiénla implementación de un **aprendizaje continuo**: Cada cambio en la base de datos corporativa nuevas tablas, modificaciones de campos o incorporación de catálogos puede enviarse **automáticamente** al motor de embeddings para actualizar la base vectorial.

Así, el sistema permanece **sincronizado** de forma **dinámica** con el entorno de datos, reduce la necesidad de reconfiguraciones manuales y garantiza que opera siempre con **información actualizada**.

8. Conclusiones y Trabajo Futuro

8.1. Resumen de logros del proyecto

Este proyecto ha demostrado con éxito la viabilidad de un asistente de datos multiagente basado en técnicas RAG. La solución desarrollada interpreta preguntas de negocio formuladas en lenguaje natural, genera consultas SQL correctas, las valida y devuelve resultados claros y trazables, todo ello orientado a usuarios no técnicos. A lo largo del desarrollo se construyó un prototipo funcional que incluye una interfaz gráfica intuitiva, una base de conocimiento vectorial y una orquestación modular de agentes. La eficacia del conjunto se validó mediante escenarios simulados utilizando la base de datos de ejemplo AdventureWorks, confirmando que el enfoque propuesto funciona en la práctica.

8.2. Limitaciones actuales

Aunque los resultados avalan la propuesta, el prototipo mantiene restricciones propias de un piloto. La validación se ha realizado sobre una única base de datos (AdventureWorks) y con un conjunto acotado de consultas; todavía no se ha ensayado en escenarios multi-fuente ni con esquemas que cambian con frecuencia. Además, el alcance funcional se ha centrado en analítica descriptiva: quedan fuera casos complejos que exigen reglas de negocio muy específicas, joins entre orígenes heterogéneos u operaciones de escritura.

La interpretación en lenguaje natural depende hoy de LLMs de terceros, lo que introduce cierta variabilidad en latencia y coste y limita el control fino sobre ese componente. La observabilidad está en fase inicial: faltan métricas sistemáticas de precisión, trazabilidad operativa avanzada y un circuito de feedback de usuario plenamente industrializado.

8.3. Posibles mejoras y extensiones

Entre las líneas de mejora se contempla una previsión de evolución del sistema, con diversos frentes que permitirán seguir ampliando sus capacidades y consolidar el progreso del proyecto.

Dos líneas especialmente relevantes son:

- **LLM-as-a-judge**: la integración de un modelo adicional encargado de evaluar la calidad de las respuestas antes de presentarlas al usuario. Este módulo funcionaría como filtro de consistencia y relevancia, reforzando la confianza en los resultados.
- **Sistema de retroalimentación**: un mecanismo que permita a los usuarios puntuar las respuestas obtenidas en distintos aspectos. De esta forma, se incorporaría la voz del usuario en el ciclo de mejora continua, aportando métricas de calidad y facilitando el ajuste fino del sistema.

De forma complementaria, se plantean las siguientes mejoras:

- **Optimización del rendimiento**: reducir el tiempo de procesamiento actual (aprox. 2 minutos por consulta contextualizada) mediante técnicas de optimización en el flujo de consultas.

- **Captura de ejemplos reales**: cuando una respuesta recibe una valoración positiva, generar automáticamente un *embedding* de la consulta y almacenarlo en la base de datos de ejemplos, enriqueciendo el contexto para futuras interacciones.
- **Memoria conversacional**: incorporar un mecanismo de persistencia que permita recordar el contexto de interacciones anteriores.
- **Ampliación multilingüe**: actualmente limitado a español e inglés, se prevé extender el soporte a más idiomas.
- **Actualización dinámica de documentos**: aprovechar las clases personalizadas ya disponibles para detectar cambios en la documentación (con funciones *upsert* y *triggers* sobre carpetas o *buckets*), de forma que el sistema reprocesa y actualiza automáticamente la base de conocimiento vectorial.
- **Controles de acceso**: implementar un sistema de permisos más granular (RLS/CLS) que limite la información visible según el rol del usuario.

En conjunto, estas mejoras permitirán un uso concurrente y transversal entre departamentos, democratizando el acceso a los datos de manera segura y ofreciendo salidas listas para negocio (texto, tabla y exportables).

8.4. Perspectiva de implantación real en empresas

El sistema desarrollado tiene un claro potencial de implantación en entornos empresariales, especialmente en organizaciones con recursos técnicos limitados o con un alto volumen de solicitudes internas de datos. Su arquitectura modular, la facilidad para integrarlo con distintas fuentes de información corporativa lo convierten en una solución muy versátil para diversos sectores.

Para la empresa, esto se traduce en decisiones más ágiles y fundamentadas. Los usuarios de negocio pueden obtener las respuestas que necesitan al instante y sin intermediarios, incrementando su productividad y reduciendo la carga de trabajo del departamento de IT. La autonomía que brinda la herramienta acelera la toma de decisiones y permite que los equipos técnicos dediquen más tiempo a proyectos estratégicos de alto valor, permitiendo un retorno de inversión rápido.

Gracias a su adaptabilidad, la implantación puede realizarse de forma progresiva. Una empresa puede comenzar desplegando la herramienta en un área piloto y, tras comprobar los resultados positivos, extender su uso a otros departamentos con confianza. Esta flexibilidad minimiza riesgos y facilita la adopción de una cultura orientada al dato en toda la organización.

En definitiva, el sistema propuesto deja de ser un mero prototipo de laboratorio para mostrarse como una herramienta lista para aportar valor real en el día a día de las empresas. Contribuye a democratizar el acceso a la información de forma segura, agiliza las consultas de negocio y posiciona a la organización a la vanguardia en el uso inteligente de sus datos.

9. Bibliografía

- [1] Cybage Software. (2024). *NL to SQL: Architecture Evolution*. Medium. https://medium.com/@genai_cybage_software/
- [2] Microsoft Tech Community (2024). *Building Intelligent Data Agents: Unleashing NL2SQL with Semantic Kernel*. https://techcommunity.microsoft.com/blog/azure-ai-foundry-blog/
- [3] *Chat2BD ai* https://chat2db.ai/
- [4] *Uber 2 query* https://www.uber.com/en-ES/blog/query-gpt/
- [5] Alation *https://www.alation.com/product/data-catalog/*
- [6] *RAG Techniques - Nir Diamant:* https://github.com/NirDiamant/RAG_Techniques/tree/main
- [7] LangChain-Academy https://github.com/langchain-ai/langchain-academy/tree/main
- [8] LangGraph https://github.com/langchain-ai/langgraph/tree/main/examples/rag
- [9] Self RAG [2310.11511] Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection
- [10] AI Engineering Book - O'Reilly AI Engineering [Book]
- [11] Gemini Embedding: Generalizable Embeddings from Gemini https://arxiv.org/abs/2503.07891v1

10. Anexos

10.1. Resumen de tecnologías utilizadas

- **Python:** Hemos utilizado Python como lenguaje principal de desarrollo por su versatilidad y su ecosistema de librerías especializadas en inteligencia artificial, procesamiento de datos y desarrollo rápido de prototipos. Su sintaxis sencilla facilita la construcción de notebooks para la fase inicial de pruebas, y la posterior consolidación en módulos permite estructurar un sistema más robusto y mantenible. La gran comunidad y documentación disponible también han sido factores clave en su elección.
- **Docker:** Para asegurar que el sistema pueda ejecutarse de manera consistente en distintos entornos, se ha utilizado Docker como herramienta de contenedorización. Esto nos permite empaquetar la aplicación junto con su configuración y dependencias, de modo que se pueda desplegar de forma sencilla y reproducible en cualquier infraestructura, reduciendo riesgos de incompatibilidad. Gracias a esta característica, el proyecto se convierte en una solución fácilmente exportable y lista para ser probada o integrada en entornos reales.
- **LangGraph:** La orquestación de agentes se ha implementado con LangGraph, una librería que permite modelar los flujos de interacción como un grafo. Esta representación visual y modular facilita el control de la lógica interna y el seguimiento de cómo cada agente contribuye al resultado final. Su integración con otras librerías de IA, como LangChain, simplifica el desarrollo y aporta flexibilidad para incorporar nuevos agentes en el futuro, lo que hace que el sistema sea escalable y adaptable a distintos escenarios.
- **Postgres:** Hemos optado por este motor de BBDD por ser una base de datos open source, lo que evita costes de licencias. Además de ser un motor robusto para ejecutar consultas SQL, sus capacidades avanzadas permiten modelar datos dimensionales eficientemente, algo clave en este proyecto.
  Su flexibilidad y el amplio ecosistema de extensiones, como las funciones para manejar JSON o datos geoespaciales, facilitan la implementación de funcionalidades futuras, como el manejo de la memoria del agente o la incorporación de datos no estructurados, sin necesidad de migrar a otra tecnología.
- **Chroma:** Hemos optado por esta BBDD vectorial por ser open source, lo que permite trabajar con embeddings de manera nativa y sin costes de licencia. Su diseño está optimizado para la búsqueda de similitud, una funcionalidad esencial en la conversión de lenguaje natural a SQL, y se integra de forma extremadamente sencilla con Python. Esta facilidad de uso y la disponibilidad de bibliotecas dentro de LangChain simplifican drásticamente la implementación del agente, permitiendo un rápido prototipado y experimentación.
- **Azure AI Foundry**: Este servicio de Microsoft permite desplegar endpoints desde los que consumir una amplia oferta de LLM's. Para nuestro proyecto, resultaron de especial interés aquellos modelos de OpenAI, porque facilita mucho la implementación de los mismos en sistemas de IA diseñados con librerías como lo es LangChain / LangGraph.La elección de usar Azure AI Foundry por sobre la API nativa de OpenAI radica en que al desplegar los modelos en este servicio, podemos especificar la región

en la que queremos hacerlo, lo que nos permite almacenar los datos dentro de territorio europeo, lo que se adecua al cumplimiento de la RGPD.
Dentro de las ubicaciones regionales disponibles en la UE, hemos seleccionado la ubicación de “swedencentral”, ya que es la que mayor oferta de modelos tiene. Mediante el siguiente enlace, se puede consultar el pricing de los diferentes modelos de OpenAI para esta región: [Azure OpenAI Service - Pricing]

- **Google Vertex AI**: Dado que nuestro sistema utiliza un modelo de embedding de Google, hemos tenido que utilizar los servicios de Vertex AI para su implementación. Es verdad que podríamos haber utilizado la Gemini API a través de Google AI Studio, pero nuevamente, por un tema de localización de datos, hemos priorizado Vertex AI, ya que de otra manera no se puede seleccionar la región de almacenamiento y procesamiento de datos. En nuestro caso, hemos parametrizado nuestro modelo de embedding para que utilice la región “europe-west1”.

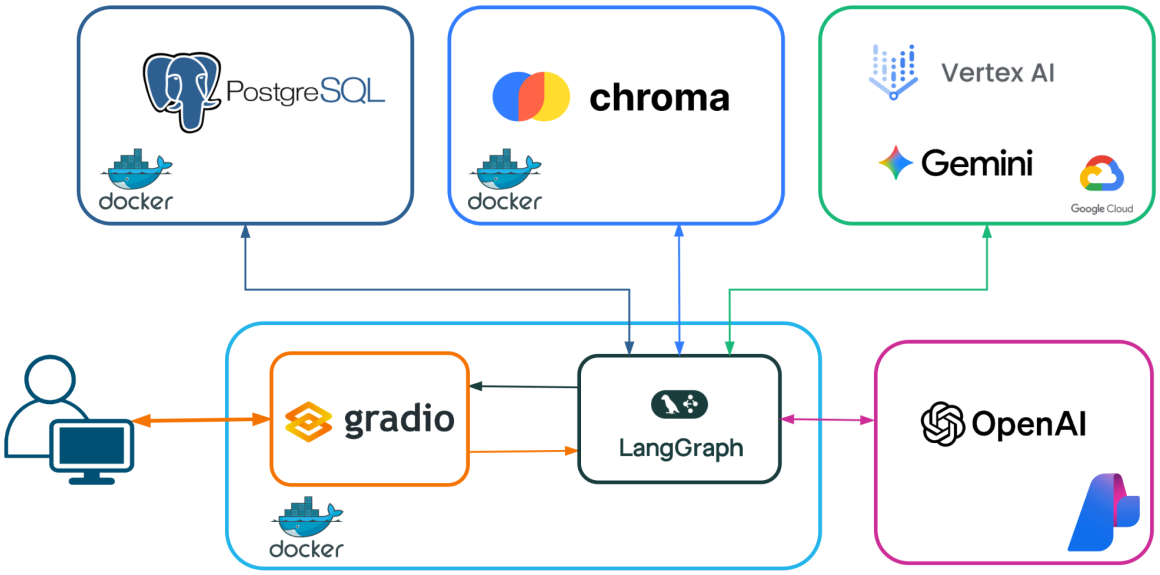
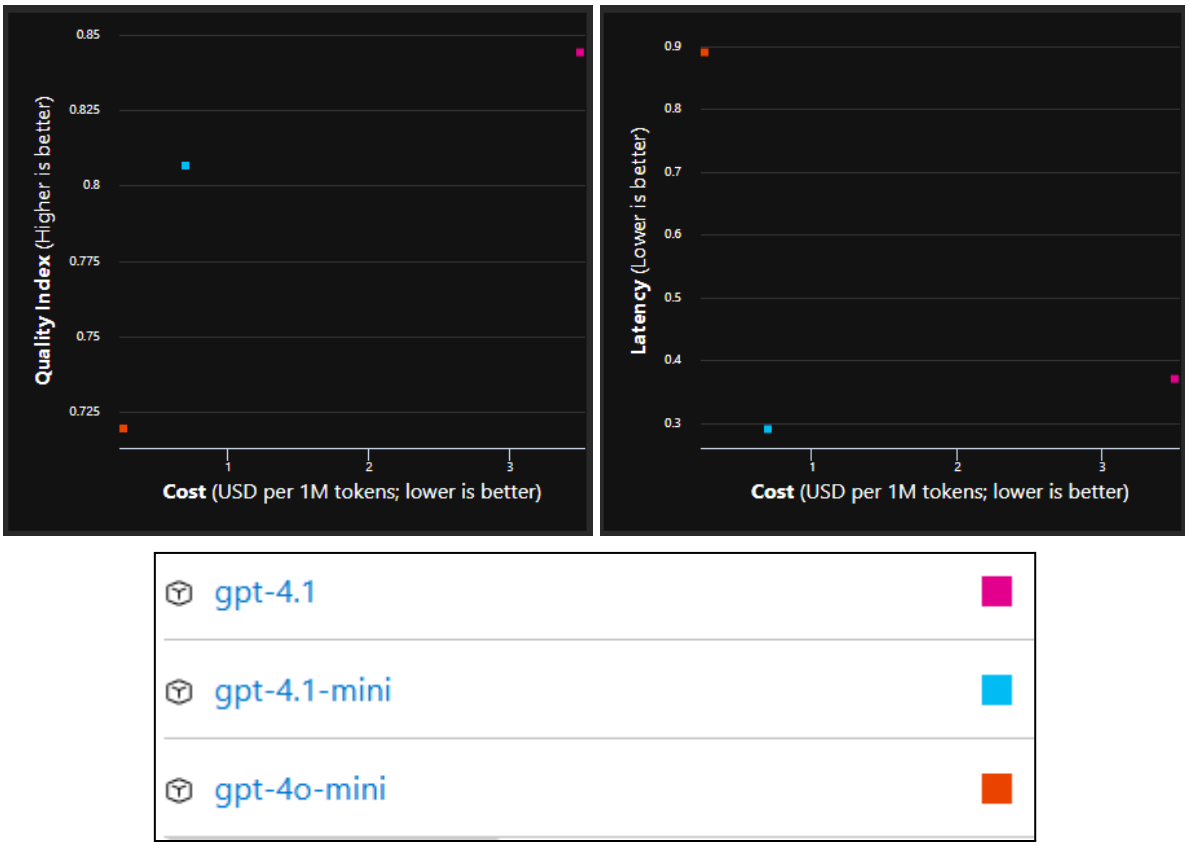


Diagrama de la distribución de las tecnologías usadas en el proyecto

10.2. Modelos de Lenguaje Utilizados (LLM's)

Como ya se ha comentado en otras secciones, en los diferentes componentes del sistema se han utilizado, en función de las necesidades específicas de las tareas a afrontar, diferentes modelos de LLM de OpenAI a través de Azure AI Foundry. Por otro lado, el modelo de embeddings que se ha utilizado para la base de datos vectorial es de Google **(gemini-embeddings-001)**, del que hablaremos específicamente en el **Anexo 10.3.1.2**. En esta sección nos centraremos en comentar los LLM's que son utilizados por nuestro sistema.

A continuación se muestra una gráfica comparativa obtenida del propio portal de Azure AI Foundry, que permite ver los modelos utilizados desde diferentes enfoques:



Los modelos de lenguaje (LLM's) que hemos utilizado en nuestro sistema son:

- ***gpt-4o-mini***: Este modelo se ha utilizado para tareas que, si bien podrían requerir de un contexto amplio y diverso, las salidas esperadas fueran acotadas. La razón principal para elegir este modelo, como puede apreciarse en las gráficas anteriores, es su reducido coste, de especial interés para tareas que se repetirán una gran cantidad de veces en nuestro sistema. A continuación, se detallan algunas tareas que han sido resueltas por este modelo:
    - Detectar el idioma de la consulta del usuario (classifier).
    - Identificar si la consulta del usuario es relevante para el alcance del sistema (relevance grader).

    - Generar subconsultas derivadas de la consulta del usuario, que permitan maximizar la recuperación de información durante la búsqueda semántica (retrievals).
    - Decidir si un extracto de texto obtenido por búsqueda semántica es realmente relevante para la consulta del usuario (relevance grader).
    - Decidir si la respuesta de un LLM generador está soportada en el contexto que se le facilitó al mismo (hallucination grader).
    - Decidir si la respuesta de un LLM generador es relevante para la consulta del usuario (answering grader).

  En el siguiente enlace se puede consultar la ficha de este modelo en Azure AI Foundry: https://ai.azure.com/catalog/models/gpt-4o-mini

- ***gpt-4.1-mini***: Este modelo se ha utilizado para tareas de generación de contenido que requiriesen de razonamiento moderadamente complejo, con un contexto amplio. La razón principal para elegir este modelo, sumada a las ya comentadas, es el excelente equilibrio que presenta entre coste, calidad y una muy baja latencia, como bien puede observarse en las gráficas anteriores. Se ha utilizado principalmente para la generación de conclusiones finales de cara al usuario, tanto de explicación de datos, como de la consulta SQL, como también la confección de gráficos.

  En el siguiente enlace se puede consultar la ficha de este modelo en Azure AI Foundry: https://ai.azure.com/catalog/models/gpt-4.1-mini

- ***gpt-4.1***: Este modelo se ha utilizado principalmente en tareas de generación de contenido que requiriesen de un razonamiento complejo, o lógica de múltiples pasos. La razón principal para elegir este modelo, es su capacidad de trabajar con un amplio contexto, presentando una muy buena calidad de resultados, a un coste y latencias razonables (menor coste que *gpt-4o*, por ejemplo). Se ha utilizado en las tareas de generación más complejas de nuestro sistema, como lo son la generación de un contexto global que contemple tanto la estructura de los datos como las reglas relevantes para negocio, y la generación y corrección de las consultas SQL.

  En el siguiente enlace se puede consultar la ficha de este modelo en Azure AI Foundry: https://ai.azure.com/catalog/models/gpt-4.1

10.3 Retrieval Augmented Generation (R.A.G.)

Nuestra base de datos vectorial la hemos implementado en Chroma y, consta de múltiples colecciones, cada una atendiendo a áreas de conocimiento bien definidas y con configuraciones específicamente seleccionadas. A lo largo de esta sección profundizaremos en ello. Todo el proceso de construcción de este componente de nuestro sistema puede ser consultado accediendo al notebook del siguiente enlace: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/notebooks/create_embeddings_in_chroma_db.ipynb

10.3.1. Colecciones - Configuración

Al momento de configurar las colecciones, se deben tomar decisiones sobre 2 aspectos principalmente, por un lado, cuál será la Función de Distancia que utilizará esa colección, y por otro, cuál será el Modelo de Embeddings con el que trabajará. A continuación discutiremos cada uno de estos aspectos:

10.3.1.1. Función de Distancia

Chroma DB permite trabajar con las siguientes funciones de distancia:

Distance	parameter	Equation	Intuition
Squared L2	`l2`	$d = \sum (A_i - B_i)^2$	measures absolute geometric distance between vectors, making it suitable when you want true spatial proximity.
Inner product	`ip`	$d = 1.0 - \sum (A_i \times B_i)$	focuses on vector alignment and magnitude, often used for recommendation systems where larger values indicate stronger preferences
Cosine similarity	`cosine`	$d = 1.0 - \frac{\sum(A_i \times B_i)}{\sqrt{\sum(A_i^2)} \cdot \sqrt{\sum(B_i^2)}}$	measures only the angle between vectors (ignoring magnitude), making it ideal for text embeddings or cases where you care about direction rather than scale

Configure - Chroma Docs

Aunque la función que la herramienta configura por defecto es la L2, para nuestras colecciones hemos decidido trabajar con la Similitud del Coseno ya que, como comenta la tabla, se prioriza la dirección por sobre la escala de los vectores. En este punto es muy importante tener en cuenta los siguientes conceptos:

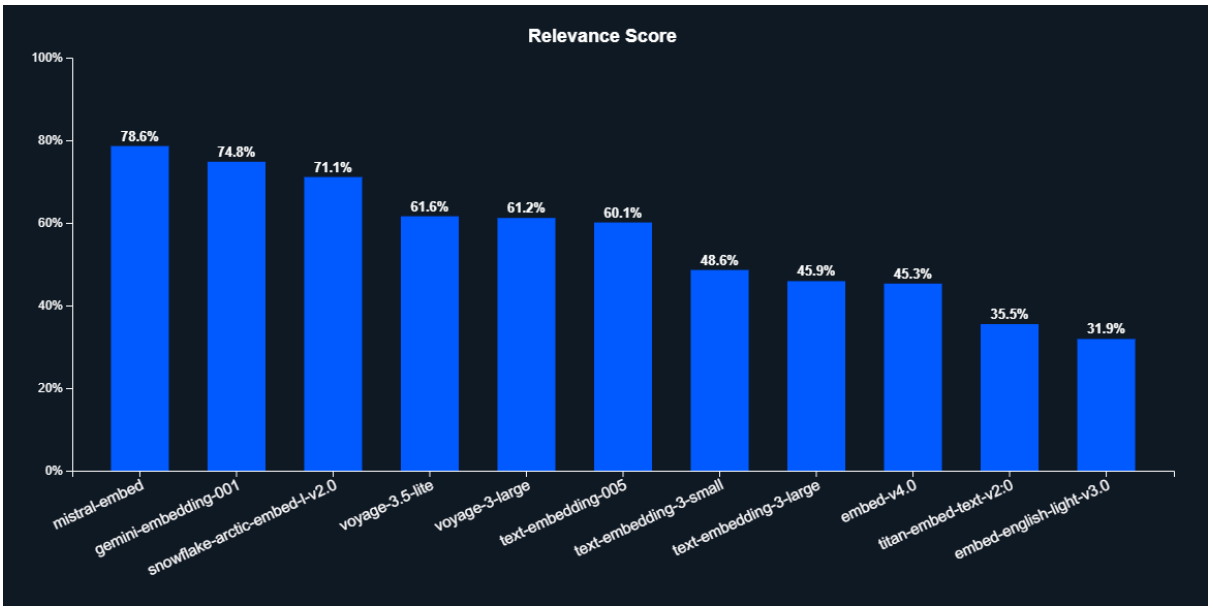
- La función de distancia que ofrece Chroma, no es la conocida Similitud del Coseno, si no que la Distancia, que , como puede verse en su expresión, se calcula como 1 - Similitud del Coseno. Es decir, esta función de distancia devuelve valores en el rango [0, 2]. Cuanto más cerca de 0, más similares serán los vectores.
- Derivado del punto anterior, al utilizar esta función de distancia, también se debe definir una función personalizada de puntaje de relevancia para evaluar nuestros resultados. Para esto, hemos tenido que definir una función personalizada, que escale los valores de distancia obtenidos en el rango [0, 1], siendo los valores más cercanos a 1 indicativo de mayor similitud entre los vectores.
- Cuando se decide trabajar con la función de Similitud del Coseno, el modelo de embedding con el que se vaya a trabajar debe devolver resultados normalizados.

10.3.1.2. Modelos de Embeddings

Durante nuestro análisis, se probaron 2 modelos de embeddings, con la misma consulta:

- **OpenAI - *text-embedding-3-large***: Los puntajes de relevancia no lograban superar los 0.70.
- **Google - *gemini-embeddings-001***: Los puntajes de relevancia conseguían superar 0.80.

Si bien estos modelos son similares, ya que ambos poseen un tamaño de embeddings de 3072 por defecto, y sus resultados se presentan normalizados, como podemos ver en el siguiente gráfico, el modelo de Google presenta resultados mucho mejores que el de OpenAI:



Embedding Models: OpenAI vs Gemini vs Cohere

Estos resultados son sustentados cuantitativamente por el paper de presentación del modelo de Google, al que se puede acceder con el enlace a continuación y muestra la siguiente comparativa:

Gemini Embedding: Generalizable Embeddings from Gemini

		Gemini Embedding	Gecko Embedding†	gte-Qwen2-7B-instruct	multilingual-e5-large-instruct	Cohere-embed-multilingual-v3.0	text-embedding-3-large
MTEB(Multilingual) (Enevoldsen et al., 2025)	Mean (Task)	**68.32**	62.13	62.51	63.23	61.10	58.92
	Mean (Type)	**59.64**	54.32	56.00	55.17	53.31	51.48
	- Bitext Mining	79.32	70.73	73.92	**80.13**	70.50	62.17
	- Classification	**71.84**	64.64	61.55	64.94	62.95	60.27
	- Clustering	**54.99**	48.47	53.36	51.54	47.61	47.49
	- Inst. Retrieval	**5.18**	4.08	4.94	-0.40	-1.89	-2.68
	- Multilabel Class.	**29.16**	22.80	25.48	22.91	22.74	22.03
	- Pair Class.	83.64	81.14	**85.13**	80.86	79.88	79.17
	- Reranking	**65.72**	61.22	65.55	62.61	64.07	63.89
	- Retrieval	**67.71**	59.68	60.08	57.12	59.16	59.27
	- STS	**79.40**	76.11	73.98	76.81	74.80	71.68
MTEB(Eng, v2) (Enevoldsen et al., 2025)	Mean (Task)	**73.30**	69.53	70.72	65.53	66.01	66.43
	Mean (Type)	**67.67**	64.82	65.77	61.21	61.43	62.15
MTEB(Code)* (Enevoldsen et al., 2025)		**74.66**	65.40	56.41	57.94	51.94	58.95
XOR-Retrieve (Asai et al., 2021)		**90.42**	65.67	N/A	N/A	N/A	68.76
XTREME-UP (Ruder et al., 2023)		**64.33**	34.97	17.39	18.68	N/A	18.80
Commercial Use		✓	✓			✓	✓

Table 1 | Comparison of embedding models on Massive Multilingual Embedding Benchmark: MTEB(Multilingual), MTEB(Eng, v2), and MTEB(Code). We also show results on XOR-Retrieve and XTREME-UP. For MTEBs, we report task and type mean performances. We report MRR@10 for XTREME-UP and Recall@5kt for XOR-Retrieve. *: Averaged over seven code tasks available for all models. †: For Gecko Embedding (Lee et al., 2024), we evaluate text-embedding-004 on MTEB(Eng, v2), text-embedding-005 on MTEB(Code), and text-multilingual-embedding-002 on others.

https://arxiv.org/abs/2503.07891v1

La diferencia sustancial de resultados en nuestro sistema es atribuíble, no solo a su mejor performance en contextos multilingües, que es lo que ocurre como veremos más adelante con parte de nuestra documentación, si no también con que los modelos de embeddings de Google optimizan su parametrización permitiendo configurar Tipos de Tareas, como se puede leer en el siguiente enlace: Choose an embeddings task type | Generative AI on Vertex AI | Google Cloud

La implementación del modelo de embeddings de Google utilizando la API de Vertex AI, para asegurar la localización de nuestros datos, en nuestro sistema se realiza utilizando la clase definida en el script que puede consultarse en el siguiente enlace: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/back/embeddings/genai.py

10.3.2. Colecciones - Áreas de Conocimiento

Nuestro sistema se compone de 3 áreas de conocimiento principalmente. Por un lado tenemos la Lógica de Negocio, y la estructura de nuestros datos (bajo el concepto de MDL), utilizados por nuestro agente generador de contexto para poder extraer información relevante para la consulta del usuario, y, por otro lado ejemplos, cuya recuperación es utilizada por el agente generador de SQL a modo de “few-shot examples”.

A continuación, brindaremos detalles de cómo se han estructurado estos tres componentes, y veremos cómo ficheros de naturalezas diferentes, requieren de enfoques diferentes para poder acometer tareas muy similares.

10.3.2.1. Esquema de Datos - MDL

Las siglas MDL, significan Modeling Definition Language, y es una de las formas de definir un modelo semántico, ya que es una representación estructurada de las tablas de nuestro esquema de base de datos, junto con metadata que permite añadir contexto tanto a ellas como a los campos que las conforman.

10.3.2.1.1. Chunking

En nuestro caso, valiéndonos de la información disponible en el INFORMATION SCHEMA de nuestra Base de Datos, hemos obtenido un fichero YAML base, que captura toda la estructura, incluyendo tipos de datos y la existencia de claves primarias y foráneas. Sobre este fichero base, luego se ha añadido metadata, indicando descripciones, tanto a nivel cabecera de tabla, como también para cada campo. A continuación, puede verse un extracto del fichero:

```yaml
database: adventure_works_dw
description: 'AdventureWorks es una base de datos de ejemplo facilitada por Microsoft. Esta base de datos admite esc
schemas:
- name: sales
  description: 'Se trata de un esquema de ventas con espíritu de simplificar la información del módulo de ventas de
  tables:
  - name: dim_customer
    description: 'Tabla de dimensión que almacena la información detallada de cada cliente que haya comprado online
    columns:
    - name: customer_key
      description: 'Clave primaria para identificar de forma única a cada cliente.'
      data_type: INT4
      is_primary_key: true

    - name: geography_key
      description: 'Clave foránea que apunta a los datos geográficos del cliente en la tabla `dim_geography`.'
      data_type: INT4
      is_foreign_key: true
      reference: sales.dim_geography.geography_key

    - name: customer_full_name
      description: 'Nombre/s y apellido/s del cliente.'
      data_type: TEXT

    - name: birth_date
      description: 'Fecha de nacimiento del cliente.'
      data_type: DATE

    - name: marital_status
      description: "Estado civil del cliente. 'M'=Married(Casada/o), 'S'=Single(Soltera/o)"
      data_type: BPCHAR(1)
```

El fichero YAML completo se puede consultar en el siguiente enlace:
https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/data/embeddings/documents/MDL_adventure_works_dw.yaml

Todo lo relacionado a la creación del fichero base, como también a su posterior chunking, que veremos a continuación, está implementado en la clase definida en el script del siguiente enlace:

https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/utils/doc_generators/mdl_generator.py

Una vez construido el fichero que recoge la estructura de la base de datos, como también metadata que brinde contexto adicional, el siguiente paso era generar los chunks, sobre los que se generarían embeddings a almacenar en nuestra base de datos Chroma DB. Este proceso representó todo un desafío, ya que debía ejecutarse un proceso de chunking de manera tal que los resultados obtenidos sean semánticamente relevantes, manteniendo un equilibrio entre el detalle a nivel de campos, y la visión global a nivel de tablas. Para esto, y dada la naturaleza del documento, se optó finalmente por un enfoque jerárquico, cuya lógica está recogida en la clase definida en el script del siguiente enlace:
https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/utils/splitters/mdl_splitter.py

Utilizando esta clase, se han creado dos colecciones, que describiremos a continuación:

mdl_tables_summary

Los chunks que conforman esta colección almacenan un resumen de las tablas, en el que se incluye la base de datos, el esquema, el nombre de la tabla, una descripción y sus claves primarias y foráneas. Adicionalmente, cada registro de la colección tenía como metadata asociada, el nombre de fichero de origen, el nombre de la base de datos, el nombre del esquema, el nombre de la tabla, y el id de esa tabla dentro de la propia colección. Toda esta metadata está pensada para el filtrado de registros al momento de querer recuperar información. A continuación se muestra un ejemplo de chunk de esta colección:

```
Database: adventure_works_dw
Schema: sales
Table: fact_sales
Table description: Tabla de hechos que contiene el detalle de las ordenes de ventas que ya han sido entregadas, con una granularidad a nivel de
línea, mostrando siempre la última versión de cada pedido. SIEMPRE que se soliciten datos de ventas, como cantidades vendidas, importe de ventas,
costes de ventas, impuestos, costes de envío, deberán ser obtenidos de esta tabla. Permite hacer desgloses a nivel de productos, clientes,
tiendas/distribuidores, división territorial, promociones aplicadas y vendedores involucrados.
Table PRIMARY KEY:
- sales_order_number
- sales_order_line_number
Table FOREIGN KEYS (Column name, Reference):
- (product_key, sales.dim_product.product_key)
- (reseller_key, sales.dim_reseller.reseller_key)
- (employee_key, sales.dim_sales_person.employee_key)
- (customer_key, sales.dim_customer.customer_key)
- (promotion_key, sales.dim_promotion.promotion_key)
- (sales_territory_key, sales.dim_sales_territory.sales_territory_key)

{'file_name': 'MDL_adventure_works_dw.yaml', 'table_id': '85e28078-2e08-4019-8ce9-3f4bf4438e1c', 'database_name': 'adventure_works_dw',
'schema_name': 'sales', 'table_name': 'fact_sales'}
```

mdl_columns

Los chunks que conforman esta colección almacenan el detalle de cada columna de cada tabla, que no fuera clave de ningún tipo (primaria ni foránea). Adicionalmente, cada registro de la colección tenía como metadata asociada, el nombre de fichero de origen, el nombre de la base de datos, el nombre del esquema, el nombre de la tabla, y el id de la tabla dentro de la colección “mdl_tables_summary”, el nombre de la columna, y su tipo de dato.Toda esta metadata está pensada para el filtrado de registros al momento de querer recuperar información. A continuación se muestra un ejemplo de chunk de esta colección:

```
Column name: ship_date
Column data type: DATE
Column description: Fecha en la que se envió el pedido al cliente. Se debe priorizar solo en los casos que se quieran calcular
tiempos de preparación de pedidos. NUNCA vendrá a NULL, dado que se trata de todos pedidos que ya han sido entregados.

{'file_name': 'MDL_adventure_works_dw.yaml', 'table_id': '85e28078-2e08-4019-8ce9-3f4bf4438e1c', 'database_name':
'adventure_works_dw', 'schema_name': 'sales', 'table_name': 'fact_sales', 'column_name': 'ship_date', 'column_data_type': 'DATE'}
```

10.3.2.1.2. Retrieval

La lógica de recuperación de registros dentro de estas colecciones sigue también un enfoque jerárquico, es decir, primero se buscan las N tablas que superen un determinado puntaje de relevancia, luego, se obtienen las M columnas pertenecientes a estas tablas que superen otro determinado puntaje de relevancia. La cantidad de tablas y columnas, como los umbrales de puntajes de relevancia no tienen porqué coincidir. La implementación de colecciones personalizadas de ChromaDB que facilitan la recuperación jerárquica para nuestro sistema se ha realizado con la clase definida en el script del siguiente enlace: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/back/chroma_collections/hierarchical.py

A continuación se muestra un ejemplo de cómo lucen los resultados obtenidos mediante este método:


```
[
  [
    {
      "table_summary": {
        "content": "Database: adventure_works_dw\nSchema: sales\nTable: fact_sales\nTable description: Tabla de hechos que contiene el detalle de las
ordenes de ventas que ya han sido entregadas, con una granularidad a nivel de línea, mostrando siempre la última versión de cada pedido. SIEMPRE que se
soliciten datos de ventas, como cantidades vendidas, importe de ventas, costes de ventas, impuestos, costes de envío, deberán ser obtenidos de esta
tabla. Permite hacer desgloses a nivel de productos, clientes, tiendas/distribuidores, división territorial, promociones aplicadas y vendedores
involucrados.\nTable PRIMARY KEY:\n- sales_order_number\n- sales_order_line_number\nTable FOREIGN KEYS (Column name, Reference):\n- (product_key,
sales.dim_product.product_key)\n- (reseller_key, sales.dim_reseller.reseller_key)\n- (employee_key, sales.dim_sales_person.employee_key)\n-
(customer_key, sales.dim_customer.customer_key)\n- (promotion_key, sales.dim_promotion.promotion_key)\n- (sales_territory_key,
sales.dim_sales_territory.sales_territory_key)",
        "relevance_score": 0.8164840936660767
      },
      "columns": [
        {
          "content": "Column name: order_quantity\nColumn data type: INT2\nColumn description: Cantidad vendida de cada producto.",
          "relevance_score": 0.8307241499423981
        },
        {
          "content": "Column name: unit_price\nColumn data type: NUMERIC\nColumn description: Precio de venta en USD de cada unidad de producto.",
          "relevance_score": 0.8191922605037689
        },
        {
```


El concepto detrás de este método de recuperación jerárquico en etapas resuelve 2 problemáticas:

1. Se logra obtener información general de la tabla, al mantener el chunk resumen de la misma, mientras que se puede obtener el detalle necesario de las columnas relevantes para la consulta del usuario.

2. Se acota la cantidad de registros entre los que la base de datos vectorial debe buscar relevancia semántica entre las columnas, ya que se acota únicamente a aquellas tablas que ya hayan cumplido con el umbral de relevancia establecido.

En el siguiente enlace se puede profundizar sobre la naturaleza y los beneficios de esta técnica:
https://github.com/NirDiamant/RAG_Techniques/blob/main/all_rag_techniques/hierarchical_indices.ipynb

10.3.2.2. Lógica de Negocio

Este componente de nuestra base de datos de conocimiento comprende una sola colección dentro de Croma DB, a la que hemos llamado “business_logic”, que contiene todas las definiciones y reglas de negocio que debe tener en cuenta nuestro sistema al momento de confeccionar una consulta SQL que cumpla con la lógica requerida por el usuario.

10.3.2.2.1. Chunking

En este caso, el documento a procesar viene en formato de texto Markdown. Si bien este tipo de documentos tiene una componente jerárquica / lógica, ya que se divide en secciones y demás, dentro de cada una de estas secciones está presente una gran cantidad de texto, lo que demanda un enfoque diferente. El documento markdown de partida puede consultarse accediendo al siguiente enlace:
https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/data/embeddings/documents/business_rules.md

En este caso, lo que se ha hecho es separar el documento en sus componentes jerárquicas y, dentro de cada una de ellas, hacer un chunking ajustado por largo de caracteres con solapamiento, asegurando así, primero una separación lógica, y luego que no se pierda contenido en la transición entre un fragmento y otro. Al hacer el chunking se ha optado por un largo de caracteres y de overlap que permita capturar conceptos específicos sin perder generalidad, potenciando así los beneficios de la técnica de retrieval que veremos en la siguiente sección. En caso de querer profundizar en el método de chunking semántico, puede consultar el siguiente enlace:
https://github.com/NirDiamant/RAG_Techniques/blob/main/all_rag_techniques/semantic_chunking.ipynb

Adicionalmente, cada chunk tendrá asociada una metadata, compuesta por el nombre del fichero, el subtítulo y la sección a la que pertenece el chunk, el overlap máximo que se ha configurado para los chunks, el id del chunk dentro de la combinación de subtítulo y sección, y la cantidad total de chunks existentes para la misma.

Toda la lógica relacionada al procesamiento del fichero markdown, y la generación de chunks a partir de él está recogida en la clase definida en el script del siguiente enlace:
https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/utils/splitters/md_splitter.py

A continuación se muestra un ejemplo de un chunk obtenido:

```
Document(id='0a9216f9-aca3-4242-80af-3c38f849da07', metadata={'headers_chunk_id': 1, 'file_name': 'business_rules.md', 'headers_comb_total': 3, 'chunk_overlap': 300, '3_section': 'N/A', '2_subtitle': '3. Métricas Clásicas del Retail y su Interpretación'}, page_content='- Cuando se habla de productos "más vendidos", sin especificar más, generalmente nos referimos a la cantidad de unidades (`order_quantity`) vendidas. También puede ser relevante medir esto por el importe facturado (`sales_amount`) para identificar productos que, aunque se vendan en menor volumen, generan más ingresos. Pero, en este último caso se especificarán términos como "de mayor facturación".\n- Cuando se refiere a clientes, tiendas/distribuidores o vendedores que "más compran/venden", esto se mide por el importe facturado (`sales_amount`) o `net sales`. Adicionalmente, se puede considerar la frecuencia de compra (para clientes y distribuidores) o el número de pedidos gestionados (para vendedores) como métricas complementarias.\n- Para el análisis de rentabilidad, la métrica clave es el margen bruto (`gross margin`) o el porcentaje de margen bruto (`gross margin percentage`), lo cual permite comprender la contribución real de cada venta o producto a las ganancias, más allá del volumen o ingreso total. Este enfoque es vital para evaluar la eficiencia operativa.\n- En el contexto de la gestión de promociones, el éxito no solo se mide por el discount_amount o average_discount_rate, sino también por el incremento en `order_quantity` o `sales_amount` directamente atribuible a la promoción. Es fundamental analizar la rentabilidad general del período promocional para determinar su efectividad.\n- Ventas B2C (Business-to-Consumer): Estas ventas se refieren a transacciones directas con el consumidor final. En el modelo, corresponden a los pedidos donde `sale_source` es igual a `\'internet_sales\'`. Puede identificarse también como la unidad de negocio encargada del comercio minorista.')
```

10.3.2.2.2. Retrieval

La lógica de recuperación de registros de esta colección tiene dos componentes que vale la pena explicar con detenimiento. Por un lado tenemos el método de búsqueda que se utiliza, que veremos en detalle a continuación, y por el otro, la técnica con que se devuelve el resultado final.

La implementación de colecciones personalizadas de ChromaDB que facilitan la recuperación mediante esta combinación de método de búsqueda y la técnica de recuperación para nuestro sistema se ha realizado con la clase definida en el script del siguiente enlace:
https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/back/chroma_collections/context_enricher.py

Método de Búsqueda - Maximal Marginal Relevance (MMR)

Dado que para este caso, a diferencia del anterior, estamos trabajando con chunks que presentan overlapping, corremos el riesgo de que nuestros resultados sean redundantes. Acá es donde toma relevancia el método de búsqueda de Relevancia Marginal Máxima (o MMR, por sus siglas en inglés).

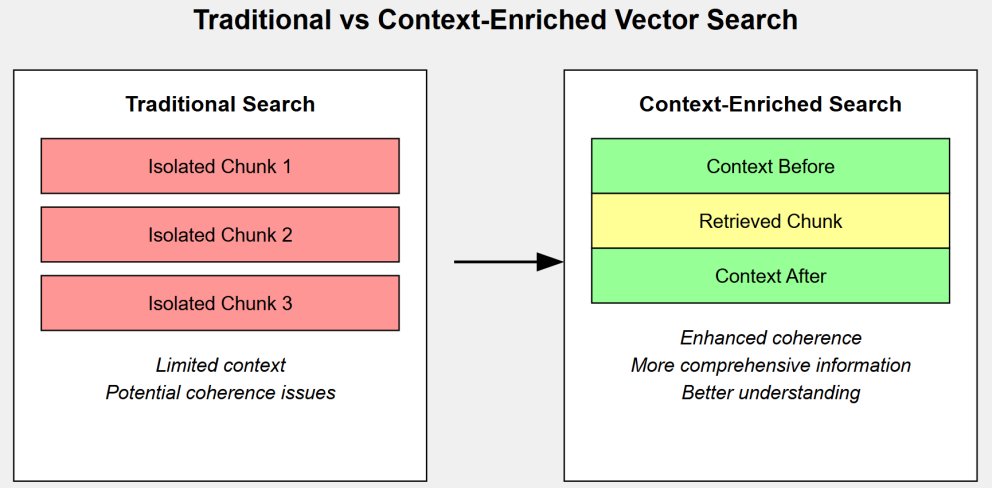
Este método de búsqueda intenta reducir la redundancia en los resultados mientras, al mismo tiempo, se mantiene la relevancia de los mismos. Es decir, un documento presenta una alta relevancia marginal si es a la vez relevante para la consulta del usuario y, además, presenta una similitud reducida con los documentos previamente seleccionados.

De esta manera, nos aseguramos de quedarnos con chunks que sean semánticamente relevantes y suficientemente diversos como para poder cubrir diferentes aspectos de la consulta del usuario.

En caso de querer profundizar sobre este método de búsqueda, se recomienda leer el siguiente paper: Summarization: (1) Using MMR for Diversity- Based Reranking and (2) Evaluating Summaries - ACL Anthology

Técnica de Recuperación - Context Enrichment Window

Esta técnica de recuperación tiene por objetivo enriquecer el contexto recuperando chunks contiguos al que ha sido obtenido como resultado de la búsqueda semántica. Esto mejora el contexto en torno a cada chunk recuperado, mejorando la coherencia y completitud de la información obtenida.



En nuestro sistema, el proceso inicia buscando chunks semánticamente relevantes para la consulta del usuario, utilizando el método MMR antes explicado, obteniendo así un muestreo con diversidad suficiente. Luego, para cada uno de los chunks recuperados, se procede a buscar los chunks contiguos, dentro de la misma jerarquía de subtítulo y sección, hasta lograr una ventana conformada por N elementos (en nuestro sistema son 3). Finalmente, se combinan todos los chunks de cada ventana, asegurando que se elimina el solapamiento existente entre ellos.

En el siguiente enlace se puede profundizar sobre la naturaleza y los beneficios de esta técnica:
https://github.com/NirDiamant/RAG_Techniques/blob/main/all_rag_techniques/context_enrichment_window_around_chunk.ipynb

10.3.2.3. Ejemplos de Consultas SQL

Este componente de nuestra base de datos de conocimiento comprende una sola colección dentro de Croma DB, a la que hemos llamado “query_examples”, que contiene ejemplos de consultas realizadas por usuarios de negocio, y las consultas SQL necesarias para responder a las mismas. Es una parte clave para el agente de generación de consultas de nuestro sistema, ya que los resultados obtenidos de esta colección se le pasan a modo de “few-shot examples” al mismo.

10.3.2.3.1. Chunking

En este caso, el documento a procesar en un JSON que contiene un listado de objetos cuyas claves son la consulta del usuario, la consulta SQL que la responde, y el idioma en que está realizada la consulta. En el siguiente enlace se puede acceder al documento completo, aunque a continuación, se muestra un ejemplo:
https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/data/embeddings/documents/query_examples.json

```
{
  "user_query": "Muéstrame los 10 productos más vendidos para el B2B en el último trimestre, excluyendo los Accesorios. Mes a
  "sql_query": "WITH \n  last_quarter AS (\n    SELECT\n      fs.product_key,\n      COALESCE(dp.spanish_product_name, dp.eng
  "language": "ES"
},
{
  "user_query": "Show me the average shipping time, delivery time, and total order processing time in days for the last quart
  "sql_query": "SELECT\n  AVG(DATE_PART('day', ship_date::TIMESTAMP - order_date::TIMESTAMP)) AS avg_shipping_time_days,\n  A
  "language": "EN"
},
```


En este caso, el chunking es mucho más sencillo, ya que solo se procederá a generar embeddings sobre la consulta del usuario, siendo el resto de campos parte de la metadata de cada registro. La metadata está compuesta entonces simplemente por el nombre del fichero, el idioma de la consulta, y la consulta SQL que responde a la consulta del usuario.

Toda la lógica relacionada al procesamiento del fichero JSON de ejemplos, y la generación de chunks a partir de él está recogida en la clase definida en el script del siguiente enlace: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/utils/splitters/json_splitter.py

10.3.2.3.2. Retrieval

La lógica de recuperación de registros de esta colección es bastante sencilla, ya que simplemente nos centraremos en buscar aquellas consultas presentes en nuestra colección de la base de datos vectorial que estén en el mismo idioma, filtrando por metadata, y que sean semánticamente similares a la consulta del usuario.

A continuación se muestra un ejemplo de resultado obtenido para una búsqueda por similitud obteniendo el top 2 de resultados más similares:

```
QUERY = 'Los 10 artículos más comprados.'

examples_results = examples_collection.search(
    queries= QUERY,
    search_type= 'similarity_score_threshold',
    k = 2
)

examples_results
```

Python


```
[(Document(id='3ae201f6-28e3-4344-b113-91cbaf511e90', metadata={'sql_query': 'SELECT\n  COALESCE(dp.spanish_product_name, dp.english_product_name) AS
producto,\n  dp.spanish_product_category_name AS categoria,\n  dp.spanish_product_subcategory_name AS subcategoria,\n  SUM(fs.sales_amount) AS importe,\n
SUM(fs.order_quantity) AS cantidad,\n  SUM(fs.sales_amount) / NULLIF(SUM(fs.order_quantity), 0) AS precio_medio\nFROM\n  sales.fact_sales fs\n  JOIN
sales.dim_product dp ON fs.product_key = dp.product_key\nWHERE\n  EXTRACT(YEAR FROM fs.order_date) = EXTRACT(YEAR FROM CURRENT_DATE) - 1\nGROUP BY\n
producto, categoria, subcategoria\nORDER BY\n  cantidad DESC\nLIMIT 10;', 'language': 'ES', 'file_name': 'query_examples.json'}, page_content='Enumera
los 10 productos más vendidos en el último año. Muestra el nombre del producto, su categoría y subcategoría, el importe de las ventas, la cantidad
vendida, y el precio medio por unidad.'),
  0.8890863955020905),
 (Document(id='1f1af88b-0381-451d-8247-2e8982f416e3', metadata={'language': 'ES', 'sql_query': 'WITH \n  last_quarter AS (\n    SELECT\n
fs.product_key,\n      COALESCE(dp.spanish_product_name, dp.english_product_name) AS product_name,\n      dp.spanish_product_category_name,\n
dp.spanish_product_subcategory_name,\n      EXTRACT(MONTH FROM fs.order_date) AS month_in_quarter,\n      fs.order_quantity\n    FROM\n
sales.fact_sales fs\n      JOIN sales.dim_product dp ON fs.product_key = dp.product_key\n    WHERE\n      fs.sale_source = 'reseller_sales'\n      AND
dp.spanish_product_category_name <> 'Accesorio'\n      AND fs.order_date >= date_trunc('quarter', CURRENT_DATE - interval '3 months')\n      AND
fs.order_date < date_trunc('quarter', CURRENT_DATE)\n  ),\n  top_products AS (\n    SELECT\n      product_key,\n      product_name,\n
spanish_product_category_name,\n      spanish_product_subcategory_name,\n      SUM(order_quantity) AS total_qty\n    FROM\n      last_quarter\n    GROUP
BY\n      product_key, product_name, spanish_product_category_name, spanish_product_subcategory_name\n    ORDER BY\n      total_qty DESC\n    LIMIT 10\n
)\n\nSELECT\n  tp.product_name AS producto,\n  tp.spanish_product_category_name AS categoria,\n  tp.spanish_product_subcategory_name AS subcategoria,\n
SUM(lq.order_quantity) FILTER (WHERE lq.month_in_quarter = EXTRACT(MONTH FROM date_trunc('quarter', CURRENT_DATE - interval '3 months'))) AS
primer_mes,\n  SUM(lq.order_quantity) FILTER (WHERE lq.month_in_quarter = EXTRACT(MONTH FROM date_trunc('quarter', CURRENT_DATE - interval '3 months') +
interval '1 month')) AS segundo_mes,\n  SUM(lq.order_quantity) FILTER (WHERE lq.month_in_quarter = EXTRACT(MONTH FROM date_trunc('quarter', CURRENT_DATE
- interval '3 months') + interval '2 month')) AS tercer_mes,\n  SUM(lq.order_quantity) AS total\nFROM\n  top_products tp\n  LEFT JOIN last_quarter lq ON
tp.product_key = lq.product_key\nGROUP BY\n  producto, categoria, subcategoria\nORDER BY\n  total DESC;", 'file_name': 'query_examples.json'},
page_content='Muéstrame los 10 productos más vendidos para el B2B en el último trimestre, excluyendo los Accesorios. Mes a mes indica cuántas unidades se
vendieron, y muestra también a qué categoría y subcategoría pertenecen.'),
  0.8772459626197815)]
```

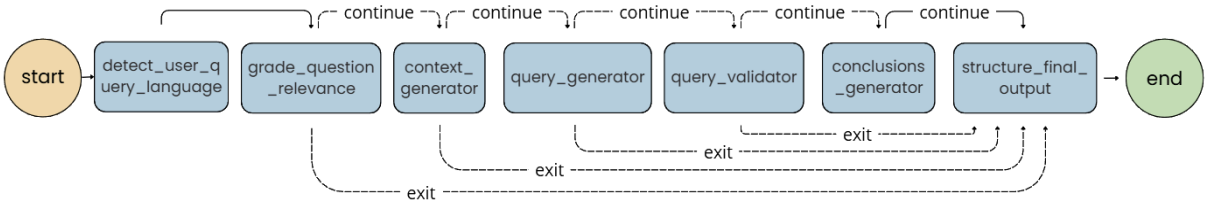

La implementación de colecciones personalizadas de ChromaDB que facilitan la recuperación de ejemplo para nuestro sistema se ha realizado con la clase definida en el script del siguiente enlace: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/back/chroma_collections/examples.py

10.4. Explicación Detallada de la Arquitectura del Sistema

En esta sección se procederá a explicar primeramente una visión global de cómo funciona nuestro sistema de agentes, y luego algunos puntos relevantes de cada uno de ellos.

La interacción del usuario con el sistema siempre se realiza a través de la UI montada en Gradio, lo que permite que lo haga de manera natural y amigable. El usuario escribe la consulta que quiere que sea respondida por el sistema, y la cantidad de frases que desea obtener en las conclusiones sobre los datos.

Esta consulta es enviada por la aplicación de Gradio a un grafo principal de LangGraph (Main Graph) que actuará como agente orquestador, manteniendo en todo momento el estado general del sistema, y facilitando la intercomunicación entre los otros agentes a través de variables de estado. A continuación se muestra un esquema del grafo en cuestión:



La primera tarea es **detectar el idioma y la relevancia** de la consulta. Si la petición no corresponde a una pregunta de negocio dentro del alcance del sistema, es decir, que pueda ser respondida con información de un data warehouse, se devuelve una respuesta informativa al usuario; en caso contrario, la solicitud se deriva al resto de agentes.

En segundo lugar, interviene el **agente generador de contexto**, que recupera información tanto de la **estructura de datos** como de la **lógica de negocio** a partir de una base de conocimiento vectorial. Para ello, descompone la consulta, ejecuta búsquedas semánticas y aplica un esquema de **Map-Reduce** para evaluar, resumir y consolidar la información más relevante, garantizando que no existan alucinaciones del modelo. El resultado es un **resumen global de contexto**, junto con los parámetros de base de datos que servirán de referencia para configurar la cadena de conexión a la misma.

A continuación, el **agente generador de queries** construye una consulta SQL en base al contexto obtenido y a ejemplos similares, validando su corrección sintáctica y seguridad. Si se genera una query válida, esta pasa al **agente validador de queries**, que comprueba la coherencia con la consulta original, la existencia de las tablas involucradas y la ejecución real contra la base de datos. En caso de errores, entra en juego una fase de **corrección de queries**, que intenta reparar la consulta hasta un máximo de 5 intentos.

Si la consulta se valida y ejecuta correctamente, el proceso continúa con el **agente generador de conclusiones**, encargado de interpretar los datos obtenidos. Este agente produce conclusiones en lenguaje natural, tanto sobre los datos obtenidos, como también sobre la estructura de la consulta SQL y, cuando es posible, genera especificaciones de gráficos siguiendo una estructura JSON que la UI puede maquetar para enriquecer la respuesta.

Finalmente, el **orquestador** centraliza los resultados, los estructura en el formato esperado por la aplicación Gradio, y marca con un indicador si la ejecución fué exitosa de manera global, o si se produjo un error en alguna etapa.

Para más información sobre las componentes gráficas de la UI, refiérase al **Anexo 10.5** donde se desarrolla el tema en profundidad.

A continuación, se deja un listado de enlaces a scripts con definiciones generales utilizadas en la construcciones de los diferentes grafos:

- Esquemas de Estados de Entrada: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/back/graphs/states/input.py
- Esquemas de Estados de Salida: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/back/graphs/states/output.py
- Implementación de Modelos: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/back/graphs/models/base.py
- Plantillas de Prompts: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/back/graphs/prompts/templates.py
- Salidas estructuradas de agentes: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/back/graphs/pydantic_models/outputs.py

10.4.1. Agente orquestador

El objetivo de este módulo es definir el flujo principal de nuestro sistema y mantener el estado y sus variables en todo momento. Actúa como el cerebro, delegando la consulta del usuario a los agentes encargados de las diferentes tareas. Puede consultar la definición de este grafo, identificado como main_graph, accediendo al script del siguiente enlace: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/back/graphs/main_graph.py

Las entradas que recibe desde la aplicación Gradio son:

- La consulta del usuario.
- La cantidad de frases de la que debe estar compuesta la conclusión que se entregue como respuesta.

Para poder mantener el control del flujo completo, adicionalmente, las variables de estado que gestiona, ya sea generando internamente, o recibiendo como respuesta del resto de los agentes, son:

- El idioma de la consulta del usuario.
- Si la pregunta del usuario muestra relevancia para el sistema (booleano).
- Si la consulta del usuario es relevante para el usuario (booleano).
- El contexto global generado.
- El nombre de la base de datos.

- El esquema de la base de datos.
- El mensaje de respuesta en caso de que no sea relevante la consulta para el contexto.
- La consulta SQL generada.
- Si la generación de la consulta se ha logrado con éxito (booleano).
- Un listado con los nombres de las tablas.
- Un listado con información de cada tabla.
- Los resultados de la consulta SQL ejecutada.
- Si se ha logrado una ejecución exitosa de la consulta SQL (booleano).
- La respuesta en lenguaje natural. Sea o bien de las conclusiones, o bien de respuestas comunicando algún problema durante la ejecución.
- La explicación de la consulta SQL en lenguaje natural.
- Un JSON con la definición de los gráficos que Gradio debe recrear.
- Si la ejecución global del sistema ha sido exitosa o no (booleano).

Tras finalizar la ejecución, este agente informa las siguientes variables a la aplicación Gradio:

- Repite la consulta del usuario.
- Repite la cantidad de frases especificadas por el usuario.
- Informa si la ejecución global ha sido exitosa o no.
- La respuesta en lenguaje natural, con conclusiones o con mensaje informando el problema.
- La consulta SQL que haya logrado generarse (sea válida o no).
- Los resultados de la consulta SQL, si logra ejecutarse.
- La explicación de la consulta SQL en lenguaje natural, si se llega a la fase de validación de conclusiones.
- El JSON con la definición de gráficos, , si se llega a la fase de validación de conclusiones y se logra generar exitosamente antes de alcanzar el límite de intentos.

Si observamos el gráfico a comienzos de la sección, podemos identificar tres nodos que pertenecen propiamente al agente orquestador. A continuación se describe cada uno de ellos:

10.4.1.1. detect_user_query_language

Es el nodo inicial del sistema. Su tarea principal es identificar el idioma de la consulta del usuario, una variable crucial para las etapas posteriores, almacenando el valor en la variable de estado.

10.4.1.2. grade_question_relevance

Realiza una verificación de relevancia para asegurar que la consulta esté relacionada con temas de negocio y que el contenido necesario se encuentre dentro del alcance de un data

warehouse empresarial. Actualiza la variable de estado con un valor booleano, siendo True que la consulta es relevante, y False que no lo es.

10.4.1.3. structure_final_output

Este es el último nodo del flujo principal. Recibe los resultados de los agentes que hayan podido participar del proceso y su función es estructurar la respuesta final, es decir, las variables de estado que hemos identificado como output, en el formato esperado por la aplicación de Gradio. En este proceso, también define un una variable booleana, que indica True si la ejecución ha sido un éxito global o False si se ha producido algún error. Una vez estructurada la respuesta, se envía para que sea percibida por el usuario.

10.4.2. Agente generador de contexto

El objetivo de este módulo es generar un resumen de contexto relevante a partir de la consulta del usuario, recuperando información de dos fuentes distintas en paralelo: la estructura de datos y la lógica de negocio. Puede consultar la definición de este subgrafo, identificado como context_generator, accediendo al script del siguiente enlace: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/back/graphs/context_generator.py

Este agente solo entrará en acción si el orquestador determina que la consulta del usuario es relevante para el objetivo del sistema.

Las entradas que recibe desde el orquestador, o grafo principal, son:

- La consulta del usuario.
- El idioma de la consulta del usuario.

Para poder mantener el control de su flujo interno, gestiona las siguientes variables de estado:

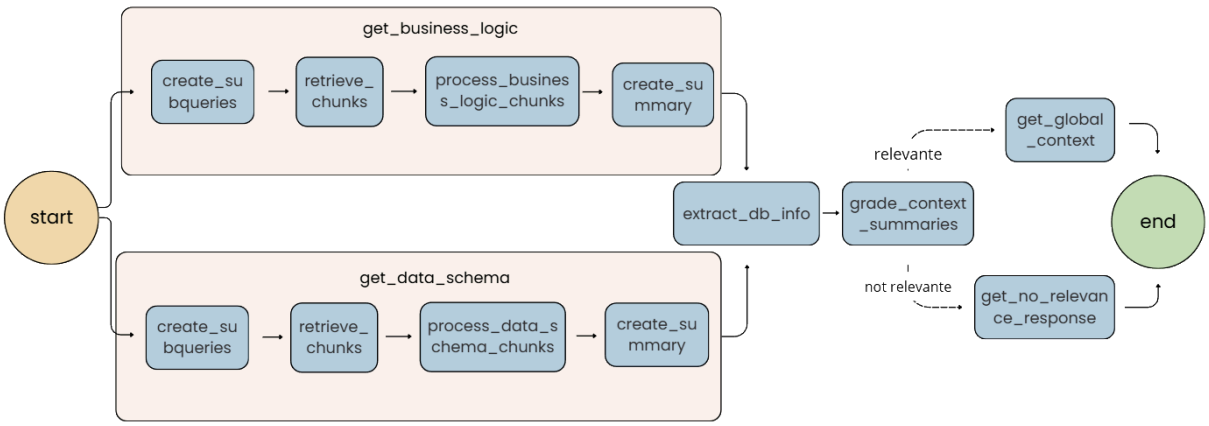
- El resumen generado correspondiente con la lógica de negocio.
- El resumen generado correspondiente con el esquema de datos.
- El nombre de la base de datos.
- El esquema de la base de datos.
- Si la consulta del usuario es relevante para el usuario (booleano).
- El contexto global generado.
- El mensaje de respuesta en caso de que no sea relevante la consulta para el contexto.
- Una lista con los resultados obtenidos para la búsqueda semántica correspondiente a la lógica de negocio.
- Una lista con los resultados obtenidos para la búsqueda semántica correspondiente al esquema de datos.

Tras finalizar la ejecución, este agente informa las siguientes variables al orquestador:

- El nombre de la base de datos.
- El esquema de la base de datos.
- Si la consulta del usuario es relevante para el usuario (booleano).

- El contexto global generado.
- El mensaje de respuesta en caso de que no sea relevante la consulta para el contexto.

A continuación, se puede observar un diagrama con los nodos que comprenden al subgrafo:



10.4.2.1. get_business_logic & get_data_schema

Ambos nodos implementan a su vez un subgrafo cada uno. Aunque dinámicamente se logran prompts diferentes para cada área de conocimiento, la estructura y la lógica de ambos subgrafos es idéntica, por lo que procederemos a explicarlo una única vez.

Estos subgrafos reciben del grafo de generación de contexto las siguientes variables:

- La consulta del usuario.
- El idioma de la consulta del usuario.
- El área de conocimiento a procesar. Esto es lo que facilita el uso de prompts dinámicos.

Internamente, gestionan las siguientes variables de estado:

- Un listado con las subqueries sobre las que se hará búsqueda semántica.
- Un listado con los resultados obtenidos de la búsqueda semántica.
- Un listado con los resúmenes de todos los documentos identificados como relevantes a criterio de un Grader LLM. Esta variable es crucial para poder llevar a cabo la etapa Map.
- El resumen del contexto específico para el área de conocimiento en cuestión, formulador sobre los resúmenes relevantes almacenados en la variable anterior. Es decir, en esta variable se almacena el resultado de la etapa Reduce.

Finalmente, informa las siguientes variables al grafo generador de contexto:

- Un listado con los resultados obtenidos de la búsqueda semántica.
- El resumen del contexto específico para el área de conocimiento en cuestión.

10.4.2.1.1. create_sub_queries

Este nodo se encarga de descomponer la consulta del usuario en hasta 5 subconsultas, haciendo que cada una de ellas se enfoque en intentar recuperar información sobre cada uno de los aspectos que componen la consulta inicial. De esta manera se busca maximizar la recuperación de información relevante.

Si desea profundizar sobre las ventajas de esta metodología de recuperación, consulte el siguiente enlace:
https://github.com/NirDiamant/RAG_Techniques/blob/main/all_rag_techniques/query_transformations.ipynb

10.4.2.1.2. retrieve_chunks

En este nodo se hace el llamado a la tool encargada de hacer la búsqueda semántica y recuperar, en simultáneo, todos los resultados relevantes para las diferentes subqueries obtenidas en el nodo anterior. Estos resultados serán únicos, es decir, que si se obtiene el mismo chunk para dos subqueries, se hará un deduplicado, que asegurará unicidad.

Es en la transición que lleva de este nodo al siguiente donde se inicia el proceso Map-Reduce.

10.4.2.1.3. process_chunks

Este nodo, nuevamente se implementa mediante un subgrafo. Ejecutándose el mismo en paralelo tantas veces como resultados únicos se hayan obtenido en la recuperación del nodo anterior.

El subgrafo recibirá las siguientes variables de estado:

- La consulta del usuario.
- El idioma de la consulta del usuario.
- El área de conocimiento a procesar. Esto es lo que facilita el uso de prompts dinámicos.
- El texto del chunk obtenido en la etapa anterior que se esté procesando.

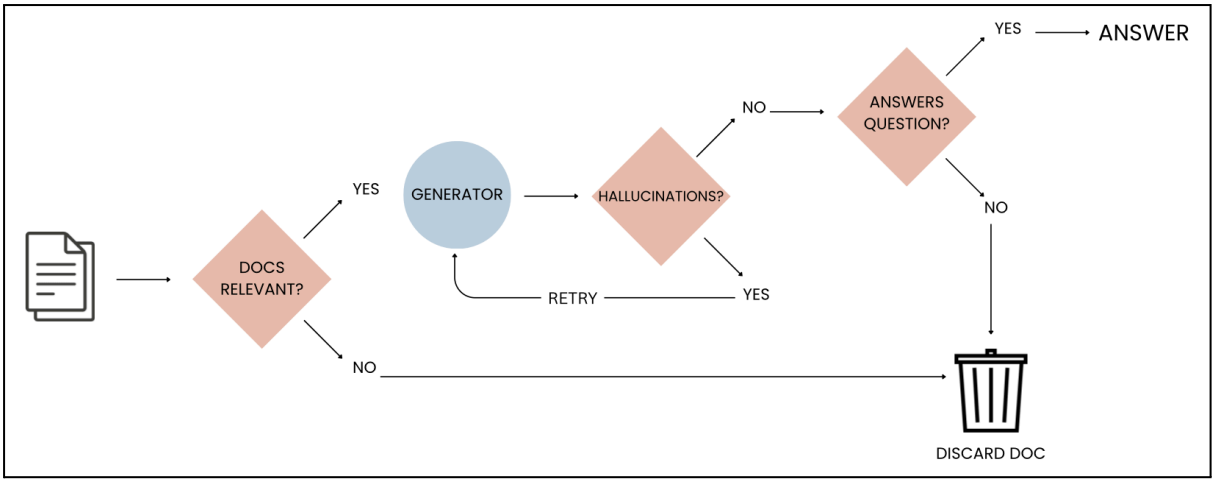
Adicionalmente, el subgrafo gestiona las siguientes variables de estado:

- Un contador de iteraciones de generación de resúmenes. Esto nos permite abortar en caso de que nos excedamos del límite configurado.
- Resumen del documento recuperado que se esté procesando, con información relevante para la consulta del usuario.

Este último valor es el que devolverá, pero dentro de una lista, y se irán añadiendo a la lista comentada dentro de las variables de estado del grafo. Esto es un requisito para poder llevar adelante el Map-Reduce en LangGraph.

En caso de querer profundizar en la implementación de Map-Reduce con LangGraph, puede consultar al siguiente enlace:
https://github.com/langchain-ai/langchain-academy/blob/main/module-4/Map-Reduce.ipynb

El proceso que se desarrolla dentro de este subgrafo, es lo que se conoce como “self-rag”. A continuación, se muestra un esquema que grafica en qué consiste:



En caso de que desee profundizar sobre la implementación de este concepto, puede consultar el siguiente enlace: https://github.com/langchain-ai/langgraph/blob/main/examples/rag/langgraph_self_rag.ipynb

Básicamente, se valida si el contenido del chunk recuperado es relevante para la consulta del usuario. Si no lo es, se descarta. Si es relevante, se genera un resumen del chunk extrayendo únicamente información relevante. Sobre ese contenido generado, se evalúa si hay alucinaciones, y si responde a la consulta del usuario. Si hay alucinaciones, se vuelve a reintentar la generación, hasta que se alcance el límite de intentos configurado.

Ya sea que se descarte por falta de relevancia, o porque el contenido generado no responde a la pregunta del usuario, aunque sea parcialmente, entonces se pasa a un nodo que fuerza una lista vacía como salida. En caso de que se pase todas las validaciones, se pasará una lista únicamente con el resumen generado.

Como comentamos anteriormente, esto es un requisito para poder implementar Map-Reduce en LangGraph.

10.4.2.1.4. create_summary

En este nodo es donde se lleva a cabo la etapa Reduce. Lo que se hace es generar un contexto unificado referido al área de conocimiento correspondiente. Para esto, se procesan todos los elementos generados en la etapa Map.

10.4.2.2. extract_db_info

En este nodo, se utiliza el resumen de contexto de esquema de datos para poder extraer del mismo el nombre de la base de datos y del esquema con el que se trabajará.

10.4.2.3. grade_context_summaries

En este nodo se evalúan los resúmenes generados sobre ambas áreas de conocimiento. Si alguno de ambos, principalmente el correspondiente al esquema de datos, muestra indicios de que no hubo resultado relevantes para la consulta del usuario, entonces actualiza la variable de estado disponible para este objetivo con un valor booleano, siendo True que existe contexto relevante para la consulta, y False si no hay evidencia de ello.

10.4.2.4. get_global_context

Este nodo sólo se alcanza en caso de que en el nodo anterior se haya actualizado la variable con valor True, es decir, haya indicios de relevancia del contexto disponible para la consulta del usuario.

En este nodo se genera el resumen final unificado de ambas áreas de conocimiento. Esto es crítico para asegurar un enfoque integral, donde se brinde un contexto no solo relevante técnicamente, si no también adecuado desde un punto de vista de negocio.

10.4.2.5. get_no_relevance_response

Este nodo sólo se alcanza en caso de que en el nodo anterior se haya actualizado la variable con valor False, es decir, no haya indicios de relevancia del contexto disponible para la consulta del usuario.

En este nodo se genera una respuesta en el idioma del usuario, indicando que actualmente no se cuenta con contexto para responder a la consulta, y se brindan ejemplos de consultas que podrían ser respondidas, basándose en los resultados obtenidos previamente durante la búsqueda vectorial.

Esto hace que los ejemplos de consultas que se pueden responder que se le dan al usuario sean representativos de la información actualizada en ese momento.

10.4.3. Agente generador de queries

El objetivo de este módulo es generar una consulta SQL válida teniendo en cuenta:

- La query del usuario en lenguaje natural.
- El contexto de negocio obtenido del agente anterior (generador de contexto).
- Ejemplos de consultas en lenguaje natural y su respectiva consulta SQL.

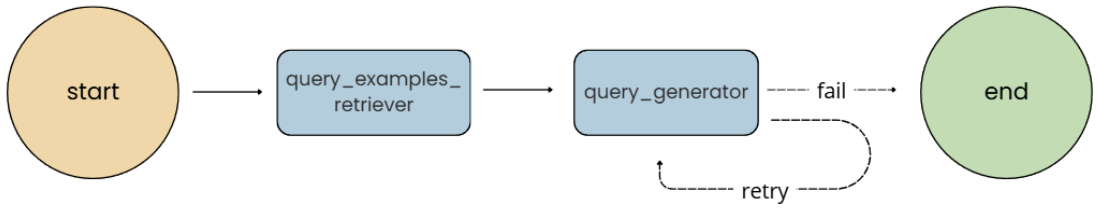
Puede consultar la definición de este subgrafo, identificado como query_generator, accediendo al script del siguiente enlace: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/back/graphs/query_generator.py

Este agente solo entrará en acción si el agente anterior determina que existe contexto suficiente y relevante para la consulta del usuario.

Este agente recibe variables del STATE global que toma como parametros de entrada, y una vez ejecutado todo el flujo, devuelve al mismo las variables correspondientes a la salida.

- variables input:
    - *context:* input con el contexto que obtuvo el agente anterior
    - *user_query:* input que contiene la consulta del usuario en lenguaje natural
- variables output:
- *valid_query_generator:* variable booleana que indica si la query generada es válida (True) o si no se puedo generar una query correctamente (False)
- *sql_query:* En caso de haber generado una query SQL válida, se contiene en esta variable

A continuación, se puede observar un diagrama con los nodos que comprenden al subgrafo:



10.4.3.1. query_examples_retriever

Es el nodo encargado de consultar a la colección 'query_examples' de Chroma, que contiene ejemplos de queries similares a las que debe de generar el próximo nodo. Utiliza la tool **get_query_examples()** que es quien realiza la llamada a la BBDD. En dicha colección hay ejemplos tanto en inglés como en español, y la tool solo devuelve aquellos que estén en el mismo idioma que la query que realizó el usuario en lenguaje natural.

El modelo de embeddings que utiliza es 'gemini-embedding-001', ya que, tras probar algunos modelos, se encontró que es el que mejor performance tiene para la búsqueda de ejemplo multilingüe.

Los ejemplos seleccionados funcionan como 'few shot examples' para el agente generador, que le permite orientarlo para generar la query SQL correcta.

10.4.3.2. query_generator

Una vez obtenida toda la información necesaria, este nodo genera una consulta SQL, utilizando el modelo 'gpt-4.1'. Paso siguiente, realiza un llamado a su tool **query_validator()** que verifica que la query generada tenga una sintaxis correcta (no se conecta a la BBDD y ejecuta la query, sino que solo corrige la sintaxis mediante la librería sqlglot) y también que la query sea solo de consulta, que no contenga ni una sentencia DML (Data Manipulation Language) ni DDL (Data Definition Language), que sí alteran el estado de la base de datos.

En caso de que la validación sea correcta, se procede a devolver la query generada al state global. Caso contrario, el flujo vuelve al nodo **query_generator,** y se vuelve a generar una nueva query SQL, teniendo en cuenta los errores encontrados anteriormente, con un máximo de N iteraciones (valor que se puede setear en el state local) en el cual ya se considera que no se puede generar una query correcta y se devuelve un error al state general.

10.4.4. Agente validador de queries

El objetivo de este módulo es asegurarse de tener conexión a la base de datos con la que se desea trabajar, validar la consulta SQL generada por el agente anterior y ejecutarla para obtener los datos necesarios. Puede consultar la definición de este subgrafo, identificado como query_generator, accediendo al script del siguiente enlace: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/back/graphs/query_validator.py

Este agente solo entrará en acción si el agente anterior consigue generar una consulta SQL que sea sintácticamente correcta.

Las entradas que recibe desde el orquestador, o grafo principal, son:

- La consulta del usuario.
- El contexto global generado.
- El nombre de la base de datos.
- El esquema de la base de datos.
- La consulta SQL generada por el agente anterior.

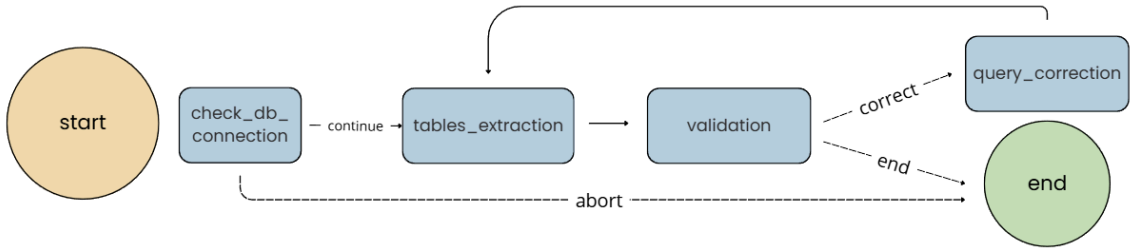
Para poder mantener el control de su flujo interno y llevar a cabo las tareas necesarias, gestiona las siguientes variables de estado:

- La cadena de conexión a la base de datos.
- Un listado con los nombres de las tablas intervinientes en la consulta SQL.
- Un listado con información relevante de las tablas que comprenden el listado anterior.
- Una lista con los resultados obtenidos tras ejecutar exitosamente la consulta SQL.
- El tipo de error que se produce al intentar validar la consulta SQL.
- El mensaje de error que se obtiene al querer validar la consulta SQL.
- La cantidad de reintentos que se llevan.
- Un indicador de si se ha logrado ejecutar la consulta SQL con éxito (booleano).

Tras finalizar la ejecución, este agente informa las siguientes variables al orquestador:

- La consulta SQL definitiva.
- El listado con los nombres de las tablas intervinientes en la consulta SQL definitiva.
- El listado con información relevante de dichas tablas.
- Los resultados obtenidos tras ejecutar exitosamente la consulta SQL definitiva.
- El indicador de si se ha logrado ejecutar la consulta SQL con éxito (booleano).

A continuación, se puede observar un diagrama con los nodos que comprenden al subgrafo:



10.4.4.1. check_db_connection

Este es el primer nodo en el subgrafo. Su función es verificar si es posible establecer una conexión satisfactoria con la base de datos y el esquema que fueron identificados por el agente generador de contexto. Si la conexión falla, el proceso se aborta y se informa al

orquestador. En caso que se logre una conexión exitosa, se almacena la cadena de conexión en las variables de estado y se avanza con la validación.

10.4.4.2. tables_extraction

Una vez que la conexión con la base de datos es exitosa, este nodo extrae un listado de todas las tablas que están involucradas en la consulta SQL generada por el agente anterior.

En este punto es crítico reforzar en el prompt que solo deben capturar las tablas, y no los nombres de las Common Table Expresions (CTE's) que pueda haber en la consulta, ya que el objetivo de este listado es confirmar la existencia de dichas tablas en el esquema de la base de datos.

10.4.4.3. validation

En este nodo se llevan a cabo 3 validaciones principalmente. La primera, es la de coherencia entre la consulta del usuario y la consulta SQL generada. Básicamente, que la consulta tenga el potencial de responder a las necesidades del usuario. En caso de que se identifique coherencia, se procede a validar la existencia de las tablas extraídas en el nodo anterior. Si se valida la existencia de todas las tablas, entonces se procede a intentar ejecutar la query.

Si alguna de las 3 validaciones anteriores falla, entonces se pasa al nodo de corrección, indicando el tipo de error, y el mensaje de error, cuando corresponda. En caso de que todo vaya bien y se logren obtener resultados, se vuelve al orquestador con la información ya indicada anteriormente.

10.4.4.4. query_correction

Este nodo se activa cuando se detecta un fallo en la coherencia o en la ejecución de la consulta. Las instrucciones para el LLM encargado de la corrección son dinámicas y se basan en el tipo de error identificado. El agente intentará corregir la consulta y luego volverá al paso de extracción de tablas para revalidar.

Este ciclo de corrección y validación se repite un máximo de 5 veces. Si después de este número de intentos no se logra una consulta válida, se informa al orquestador que la validación ha fallado. Si la corrección es exitosa, la consulta actualizada y los datos obtenidos se envían al orquestador.

10.4.5. Agente generador de conclusiones

El objetivo de este módulo es generar conclusiones en lenguaje natural, y si es posible, gráficos, a partir de los datos obtenidos de la consulta SQL. Puede consultar la definición de este subgrafo, identificado como context_generator, accediendo al script del siguiente enlace:
https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/src/back/graphs/context_generator.py

Este agente solo entrará en acción si el agente anterior consigue validar y ejecutar satisfactoriamente la consulta SQL.

Las entradas que recibe desde el orquestador, o grafo principal, son:

- La consulta del usuario.
- La cantidad de frases de la que debe estar compuesta la conclusión que se entregue como respuesta.
- El idioma de la consulta del usuario.
- La consulta SQL ejecutada por el agente anterior.
- Los resultados obtenidos tras ejecutar la consulta SQL.

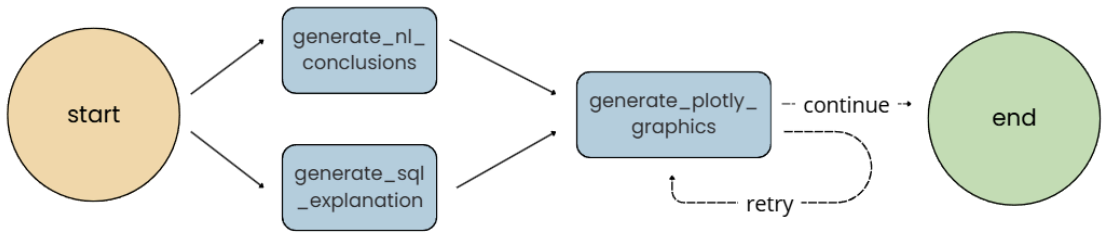
Para poder mantener el control de su flujo interno y llevar a cabo las tareas necesarias, gestiona las siguientes variables de estado:

- La cantidad de intentos de generación de gráficos.
- Las conclusiones extraídas de los datos obtenidos en lenguaje natural.
- La explicación de cómo se estructura la lógica de la consulta SQL en lenguaje natural.
- El JSON con la definición de los gráficos que deben ser maquetados por la aplicación Gradio.

Tras finalizar la ejecución, este agente informa las siguientes variables al orquestador:

- Las conclusiones extraídas de los datos obtenidos en lenguaje natural.
- La explicación de cómo se estructura la lógica de la consulta SQL en lenguaje natural.
- El JSON con la definición de los gráficos que deben ser maquetados por la aplicación Gradio.

A continuación, se puede observar un diagrama con los nodos que comprenden al subgrafo:



10.4.5.1. generate_nl_conclusions

Este nodo opera en paralelo con el generador de la explicación de la consulta SQL. Su tarea es crear un resumen en lenguaje natural de los datos obtenidos, que responda de manera clara y concisa a la pregunta original del usuario. Este resumen se debe componer de tantos elementos (cantidad) como haya especificado el usuario en la aplicación de Gradio antes de lanzar la consulta.

10.4.5.2. generate_sql_explanation

Este nodo opera en paralelo con el generador de las conclusiones en lenguaje natural. Su tarea es crear una explicación sobre cómo la consulta SQL validada llegó a la respuesta generada por el otro nodo, proporcionando transparencia al proceso.

10.4.5.3. generate_plotly_graphics

Este nodo intenta generar un JSON que, al ser procesado por una función personalizada, se traduce en un gráfico de Plotly. Si la generación del JSON no es válida para el sistema, el agente lo intentará de nuevo. Este ciclo se repite un máximo de 3 veces. Si después de los intentos el JSON sigue siendo inválido, el agente desiste y sólo devuelve las conclusiones y la explicación en lenguaje natural, sin informar de ningún error.

10.5. Gradio

La interfaz de usuario está diseñada bajo un enfoque simple y progresivo. Su objetivo principal es guiar al usuario a través del proceso de consulta de datos de una manera intuitiva, mostrando solo los elementos relevantes en cada etapa. El diseño se basa en una estructura de pestañas y columnas, lo que permite una clara separación de las funcionalidades: la entrada de la consulta, la visualización de los resultados principales y el acceso a los detalles técnicos y visuales.

La estética es limpia y funcional, priorizando la claridad del texto y la visibilidad de los datos. Elementos como los botones de "Consultar" y "Nueva consulta" tienen un papel central en la interacción, mientras que una barra lateral opcional se encarga de funcionalidades secundarias como la exportación de datos, manteniendo la pantalla principal despejada. El uso de pestañas para "Gráficos" y "SQL" ofrece al usuario la posibilidad de profundizar en la información de interés de manera organizada.

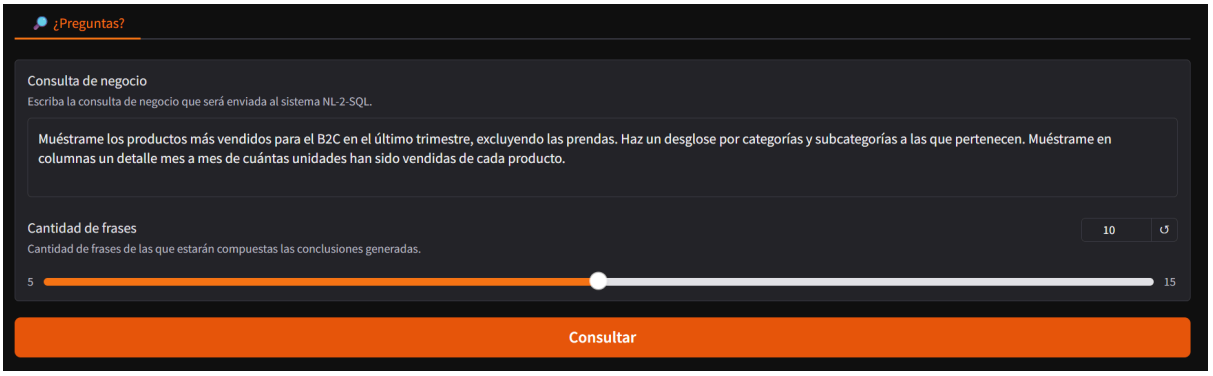
Para ver el diseño de la aplicación de Gradio, puede referirse al script del siguiente enlace: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/gradio_interface.py

10.5.1. Estructura de la interfaz

La aplicación se organiza en una estructura jerárquica y modular para facilitar la navegación:

Pestañas principales

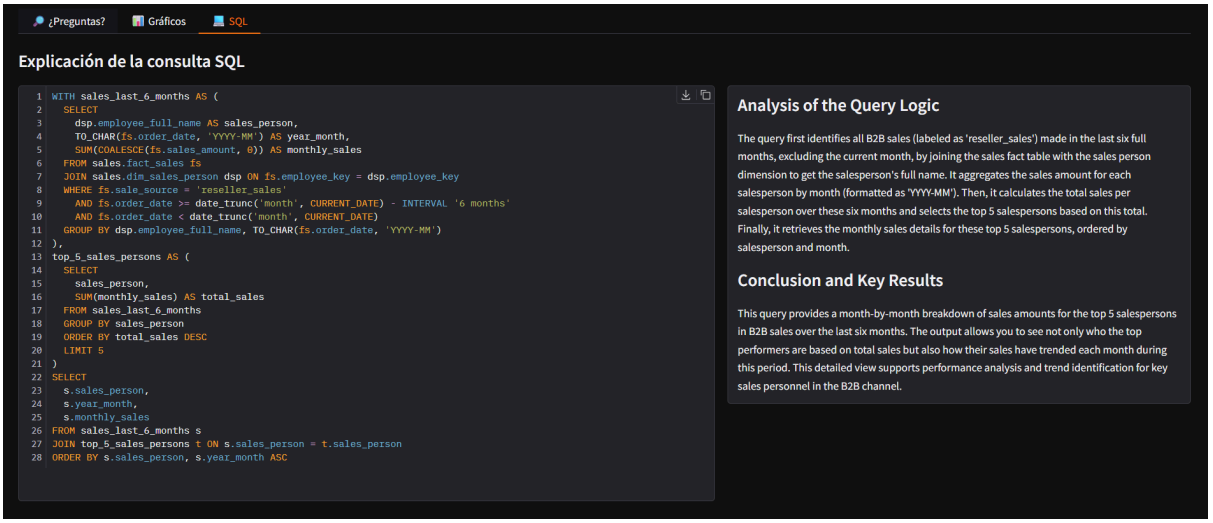
🔎**¿Preguntas?** (Pestaña de Consulta): Es la pantalla inicial donde se introduce la consulta de negocio.



📊**Gráficos**: Se activa dinámicamente si los resultados de la consulta son aptos para visualización, mostrando los gráficos correspondientes.



💻**SQL**: Se activa tras una consulta exitosa, mostrando el código SQL generado y una explicación en lenguaje natural.



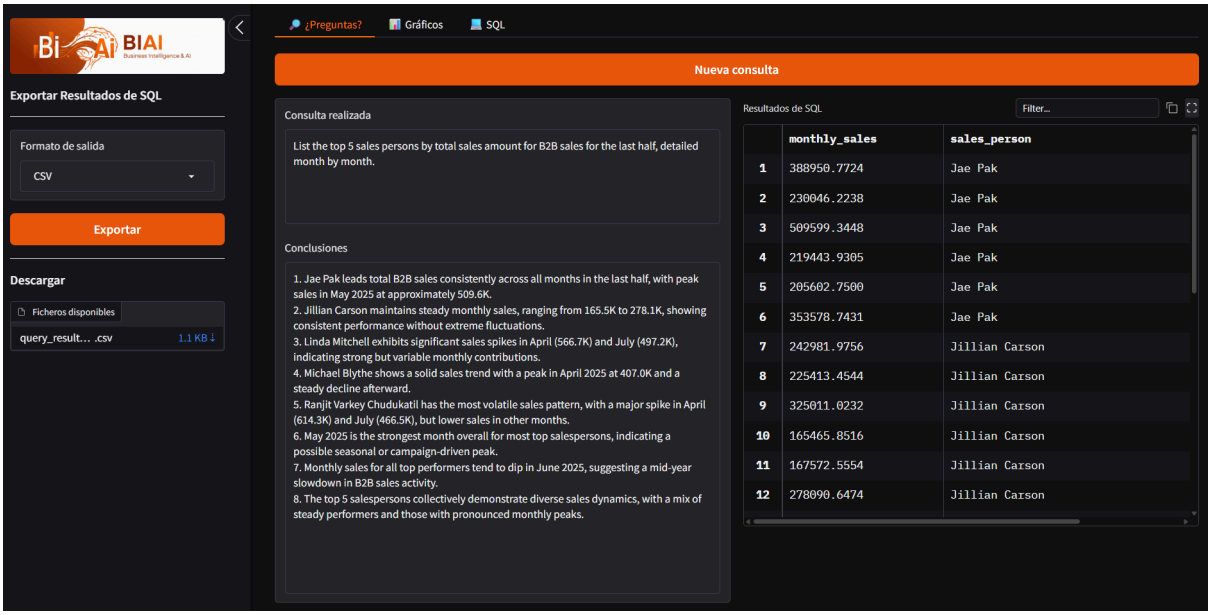
Secciones y Columnas

Sección de entrada de consulta: Contiene el campo de texto para la pregunta del usuario y el deslizador de "Cantidad de frases". Más arriba se muestra una captura de la misma.

Sección de resultados: Se vuelve visible después de una consulta y presenta las conclusiones en texto, la tabla de datos, y el botón para una nueva consulta. Más abajo se muestra una captura con la barra lateral desplegada.

Barra lateral (Sidebar)

Opciones de exportación: Contiene un menú desplegable para seleccionar el formato de exportación (CSV, Excel) y un botón de descarga. Esta barra lateral se muestra u oculta en función de si hay resultados exitosos para exportar.



Componentes de visualización de datos

- **Cuadros de texto**: Se utilizan para mostrar la consulta original y las conclusiones en lenguaje natural.
- **Tabla de datos (DataFrame)**: Presenta los resultados tabulares de la consulta SQL ejecutada.
- **Plot**: Un componente dedicado a mostrar los gráficos de Plotly generados tras el procesamiento del JSON generado por el sistema.
- **Code**: Un bloque de código diseñado para mostrar la consulta SQL ejecutada de manera legible.
- **Markdown**: Se usa para formatear la explicación de la consulta SQL, permitiendo un texto enriquecido.

Esta estructura asegura que la interacción del usuario sea intuitiva y eficiente, permitiéndole centrarse en la pregunta de negocio y en la interpretación de los resultados, sin verse abrumado por detalles técnicos innecesarios.

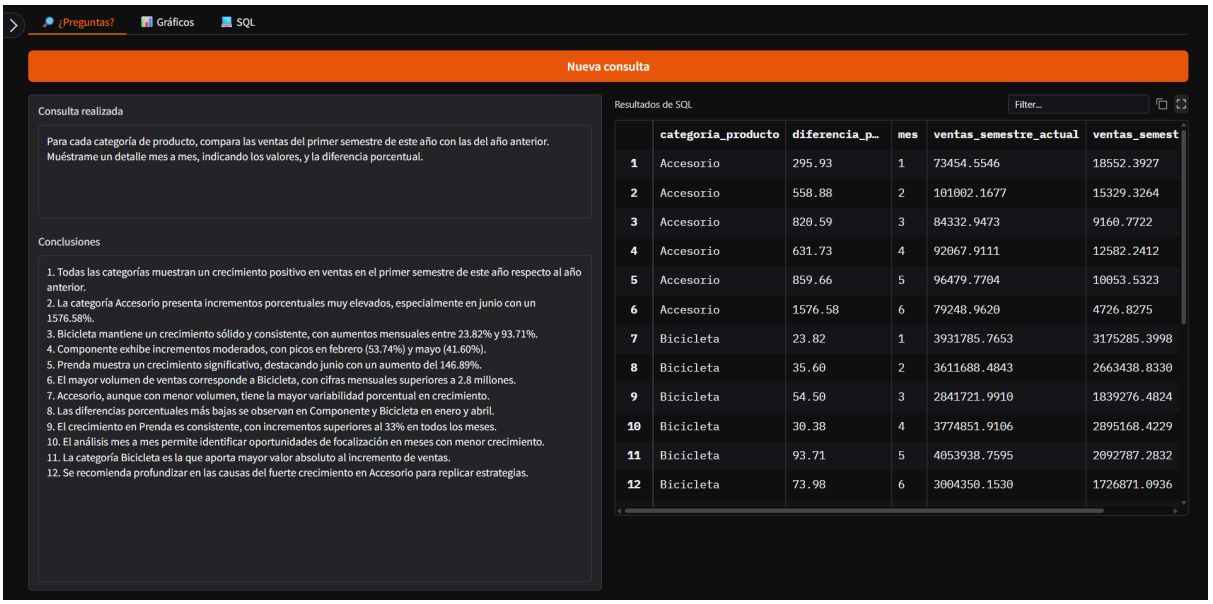
10.5.2. Ejemplos

10.5.2.1. Ejemplo 1

Consulta realizada

Para cada categoría de producto, compara las ventas del primer semestre de este año con las del año anterior. Muéstrame un detalle mes a mes, indicando los valores, y la diferencia porcentual.





Conclusiones

1. Todas las categorías muestran un crecimiento positivo en ventas en el primer semestre de este año respecto al año anterior.
2. La categoría Accesorio presenta incrementos porcentuales muy elevados, especialmente en junio con un 1576.58%.
3. Bicicleta mantiene un crecimiento sólido y consistente, con aumentos mensuales entre 23.82% y 93.71%.
4. Componente exhibe incrementos moderados, con picos en febrero (53.74%) y mayo (41.60%).
5. Prenda muestra un crecimiento significativo, destacando junio con un aumento del 146.89%.

6. El mayor volumen de ventas corresponde a Bicicleta, con cifras mensuales superiores a 2.8 millones.
7. Accesorio, aunque con menor volumen, tiene la mayor variabilidad porcentual en crecimiento.
8. Las diferencias porcentuales más bajas se observan en Componente y Bicicleta en enero y abril.
9. El crecimiento en Prenda es consistente, con incrementos superiores al 33% en todos los meses.
10. El análisis mes a mes permite identificar oportunidades de focalización en meses con menor crecimiento.
11. La categoría Bicicleta es la que aporta mayor valor absoluto al incremento de ventas.
12. Se recomienda profundizar en las causas del fuerte crecimiento en Accesorio para replicar estrategias.



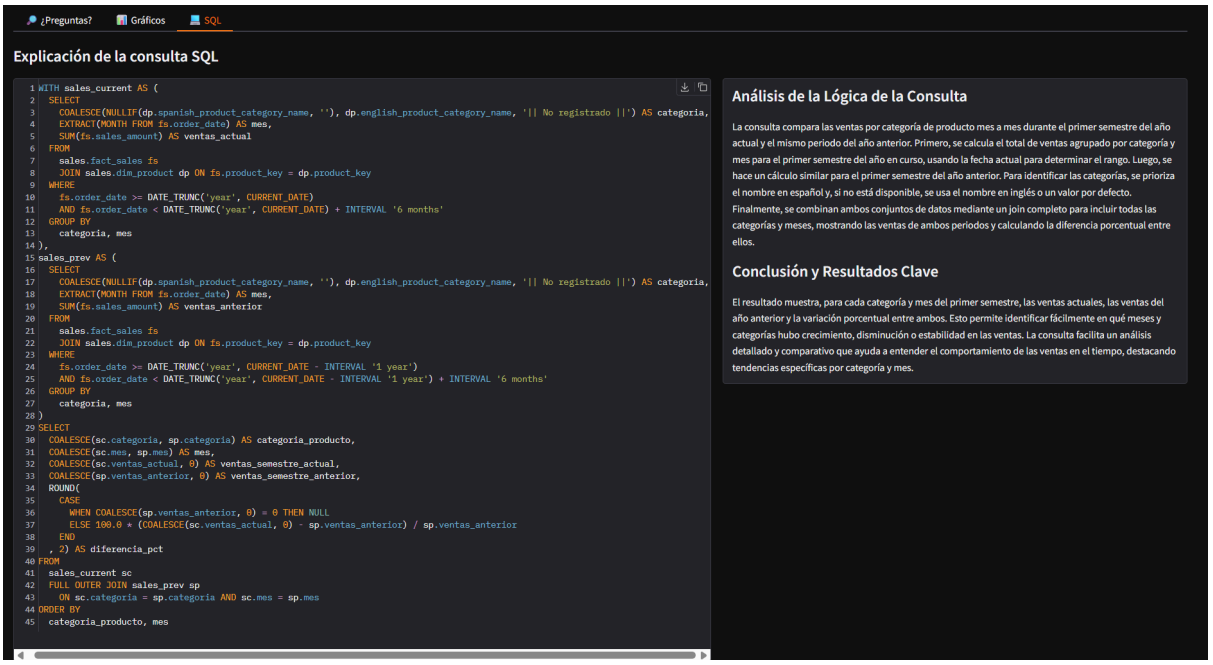
Explicación de Query

Análisis de la Lógica de la Consulta

La consulta compara las ventas por categoría de producto mes a mes durante el primer semestre del año actual y el mismo periodo del año anterior. Primero, se calcula el total de ventas agrupado por categoría y mes para el primer semestre del año en curso, usando la fecha actual para determinar el rango. Luego, se hace un cálculo similar para el primer semestre del año anterior. Para identificar las categorías, se prioriza el nombre en español y, si no está disponible, se usa el nombre en inglés o un valor por defecto. Finalmente, se combinan ambos conjuntos de datos mediante un join completo para incluir todas las categorías y meses, mostrando las ventas de ambos periodos y calculando la diferencia porcentual entre ellos.

Conclusión y Resultados Clave

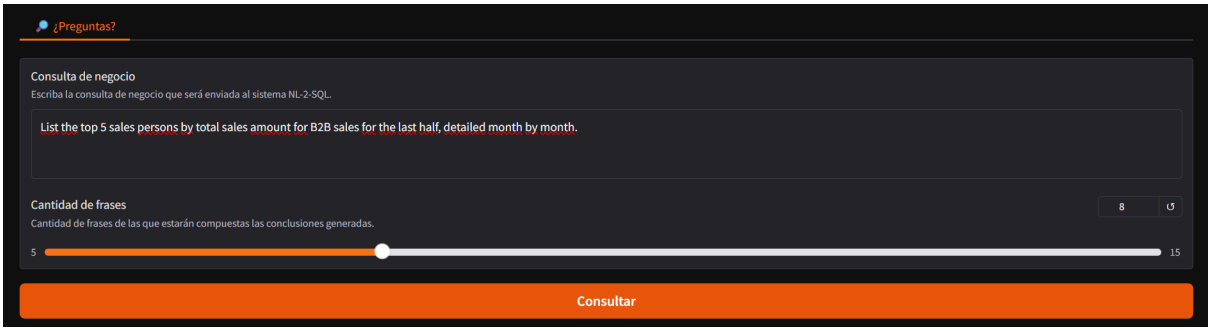
El resultado muestra, para cada categoría y mes del primer semestre, las ventas actuales, las ventas del año anterior y la variación porcentual entre ambos. Esto permite identificar fácilmente en qué meses y categorías hubo crecimiento, disminución o estabilidad en las ventas. La consulta facilita un análisis detallado y comparativo que ayuda a entender el comportamiento de las ventas en el tiempo, destacando tendencias específicas por categoría y mes.

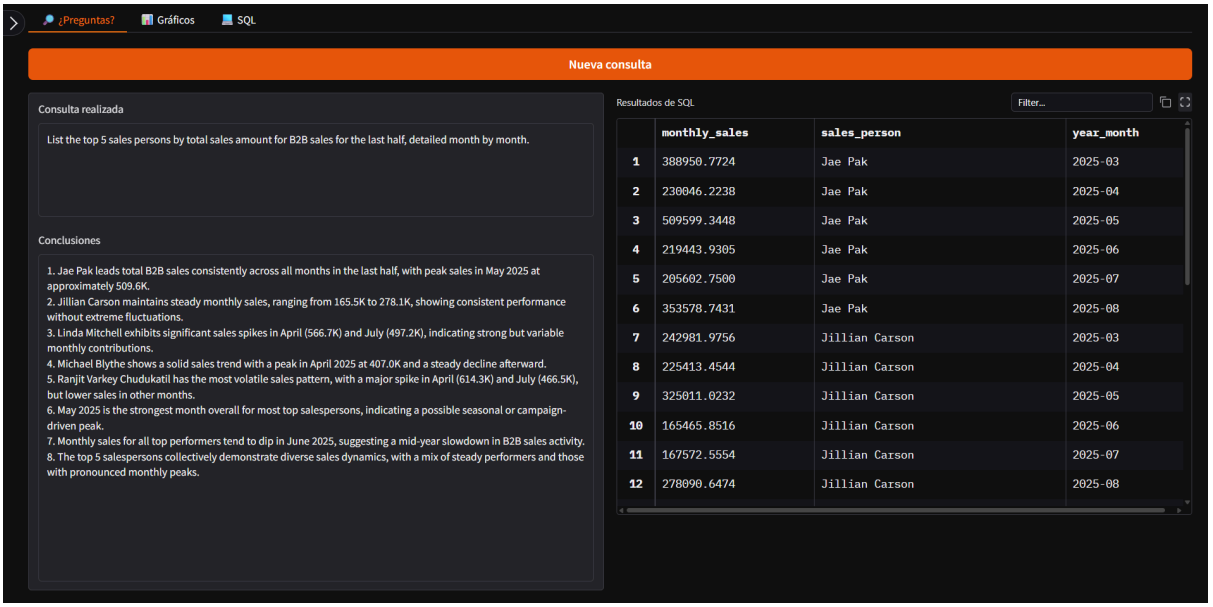


10.5.2.2. Ejemplo 2

Consulta realizada

List the top 5 sales persons by total sales amount for B2B sales for the last half, detailed month by month.





Conclusiones

1. Jae Pak leads total B2B sales consistently across all months in the last half, with peak sales in May 2025 at approximately 509.6K.
2. Jillian Carson maintains steady monthly sales, ranging from 165.5K to 278.1K, showing consistent performance without extreme fluctuations.
3. Linda Mitchell exhibits significant sales spikes in April (566.7K) and July (497.2K), indicating strong but variable monthly contributions.
4. Michael Blythe shows a solid sales trend with a peak in April 2025 at 407.0K and a steady decline afterward.
5. Ranjit Varkey Chudukatil has the most volatile sales pattern, with a major spike in April (614.3K) and July (466.5K), but lower sales in other months.
6. May 2025 is the strongest month overall for most top salespersons, indicating a possible seasonal or campaign-driven peak.
7. Monthly sales for all top performers tend to dip in June 2025, suggesting a mid-year slowdown in B2B sales activity.
8. The top 5 salespersons collectively demonstrate diverse sales dynamics, with a mix of steady performers and those with pronounced monthly peaks.



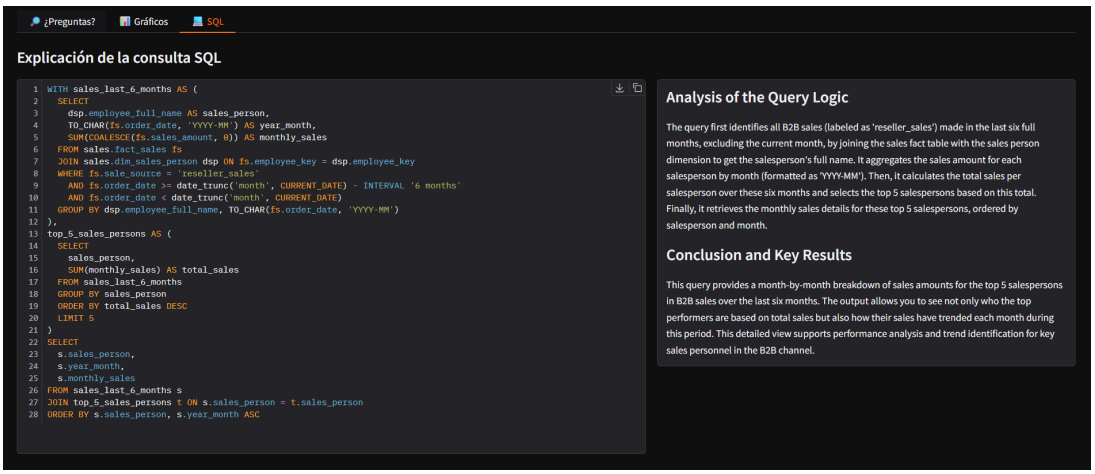
Explicación de Query

Analysis of the Query Logic

The query first identifies all B2B sales (labeled as 'reseller_sales') made in the last six full months, excluding the current month, by joining the sales fact table with the sales person dimension to get the salesperson's full name. It aggregates the sales amount for each salesperson by month (formatted as 'YYYY-MM'). Then, it calculates the total sales per salesperson over these six months and selects the top 5 salespersons based on this total. Finally, it retrieves the monthly sales details for these top 5 salespersons, ordered by salesperson and month.

Conclusion and Key Results

This query provides a month-by-month breakdown of sales amounts for the top 5 salespersons in B2B sales over the last six months. The output allows you to see not only who the top performers are based on total sales but also how their sales have trended each month during this period. This detailed view supports performance analysis and trend identification for key sales personnel in the B2B channel.

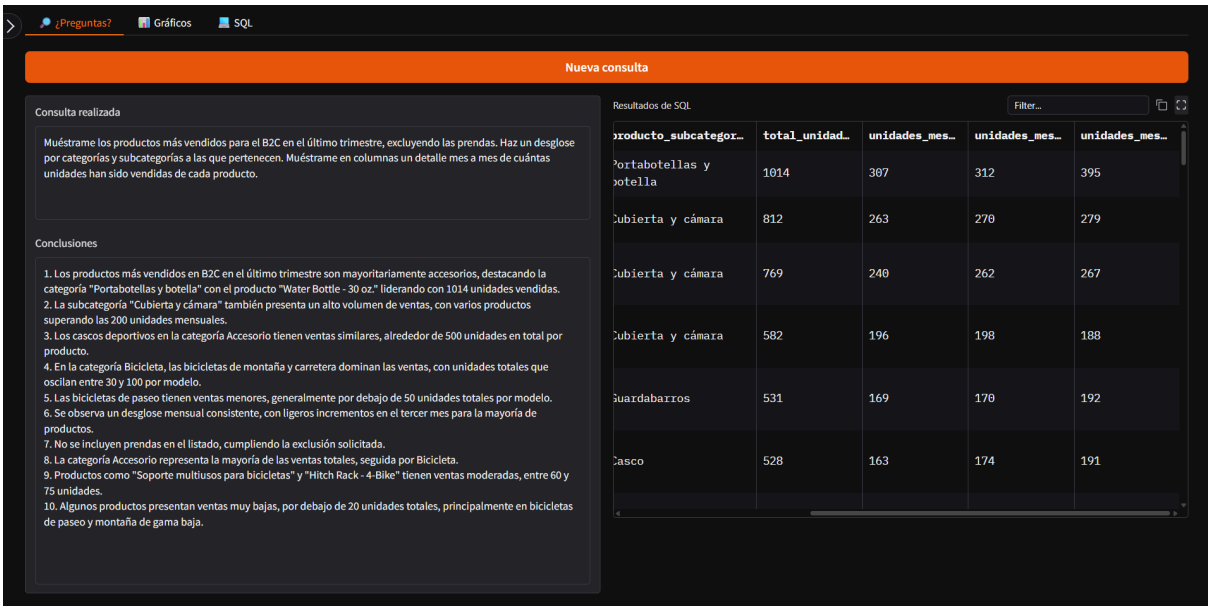


10.5.2.3. Ejemplo 3

Consulta realizada

Muéstrame los productos más vendidos para el B2C en el último trimestre, excluyendo las prendas. Haz un desglose por categorías y subcategorías a las que pertenecen. Muéstrame en columnas un detalle mes a mes de cuántas unidades han sido vendidas de cada producto.





Conclusiones

1. Los productos más vendidos en B2C en el último trimestre son mayoritariamente accesorios, destacando la categoría "Portabotellas y botella" con el producto "Water Bottle - 30 oz." liderando con 1014 unidades vendidas.
2. La subcategoría "Cubierta y cámara" también presenta un alto volumen de ventas, con varios productos superando las 200 unidades mensuales.
3. Los cascos deportivos en la categoría Accesorio tienen ventas similares, alrededor de 500 unidades en total por producto.
4. En la categoría Bicicleta, las bicicletas de montaña y carretera dominan las ventas, con unidades totales que oscilan entre 30 y 100 por modelo.

5. Las bicicletas de paseo tienen ventas menores, generalmente por debajo de 50 unidades totales por modelo.
6. Se observa un desglose mensual consistente, con ligeros incrementos en el tercer mes para la mayoría de productos.
7. No se incluyen prendas en el listado, cumpliendo la exclusión solicitada.
8. La categoría Accesorio representa la mayoría de las ventas totales, seguida por Bicicleta.
9. Productos como "Soporte multiusos para bicicletas" y "Hitch Rack - 4-Bike" tienen ventas moderadas, entre 60 y 75 unidades.
10. Algunos productos presentan ventas muy bajas, por debajo de 20 unidades totales, principalmente en bicicletas de paseo y montaña de gama baja.



Explicación de Query

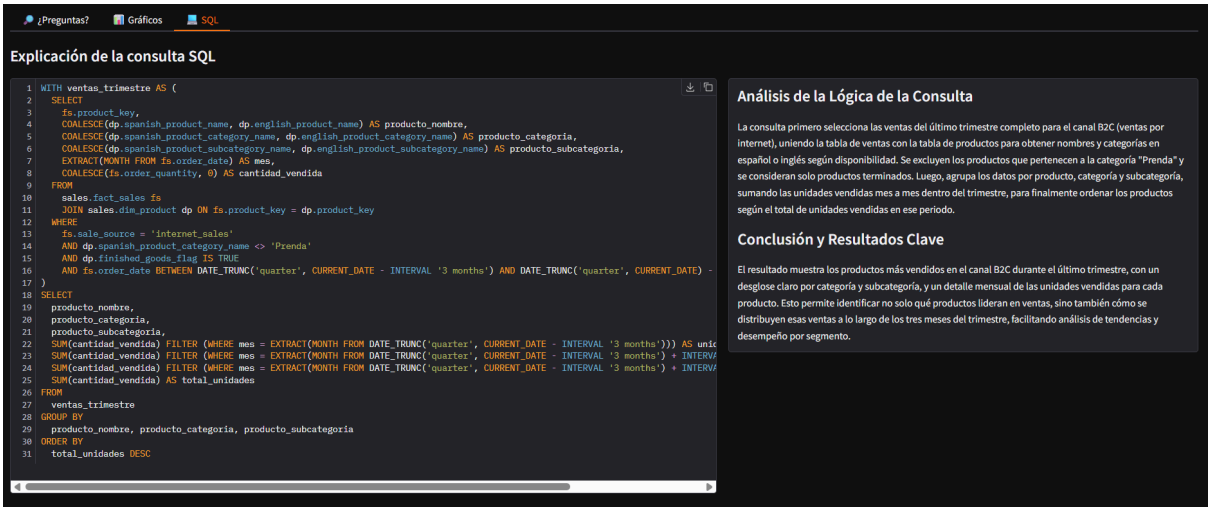
Análisis de la Lógica de la Consulta

La consulta primero selecciona las ventas del último trimestre completo para el canal B2C (ventas por internet), uniendo la tabla de ventas con la tabla de productos para obtener nombres y categorías en español o inglés según disponibilidad. Se excluyen los productos que pertenecen a la categoría "Prenda" y se consideran solo productos terminados. Luego, agrupa los datos por producto, categoría y subcategoría, sumando las unidades vendidas mes a mes dentro del trimestre, para finalmente ordenar los productos según el total de unidades vendidas en ese periodo.

Conclusión y Resultados Clave

El resultado muestra los productos más vendidos en el canal B2C durante el último trimestre, con un desglose claro por categoría y subcategoría, y un detalle mensual de las unidades vendidas para cada producto. Esto permite identificar no solo qué productos

lideran en ventas, sino también cómo se distribuyen esas ventas a lo largo de los tres meses del trimestre, facilitando análisis de tendencias y desempeño por segmento.




```sql
WITH ventas_trimestre AS (
  SELECT
    fs.product_key,
    COALESCE(dp.spanish_product_name, dp.english_product_name) AS producto_nombre,
    COALESCE(dp.spanish_product_category_name, dp.english_product_category_name) AS producto_categoria,
    COALESCE(dp.spanish_product_subcategory_name, dp.english_product_subcategory_name) AS producto_subcategoria,
    EXTRACT(MONTH FROM fs.order_date) AS mes,
    COALESCE(fs.order_quantity, 0) AS cantidad_vendida
  FROM
    sales.fact_sales fs
    JOIN sales.dim_product dp ON fs.product_key = dp.product_key
  WHERE
    fs.sale_source = 'internet_sales'
    AND dp.spanish_product_category_name <> 'Prenda'
    AND dp.finished_goods_flag IS TRUE
    AND fs.order_date BETWEEN DATE_TRUNC('quarter', CURRENT_DATE - INTERVAL '3 months') AND DATE_TRUNC('quarter', CURRENT_DATE) -
)
SELECT
  producto_nombre,
  producto_categoria,
  producto_subcategoria,
  SUM(cantidad_vendida) FILTER (WHERE mes = EXTRACT(MONTH FROM DATE_TRUNC('quarter', CURRENT_DATE - INTERVAL '3 months'))) AS uni
  SUM(cantidad_vendida) FILTER (WHERE mes = EXTRACT(MONTH FROM DATE_TRUNC('quarter', CURRENT_DATE - INTERVAL '3 months') + INTERVA
  SUM(cantidad_vendida) FILTER (WHERE mes = EXTRACT(MONTH FROM DATE_TRUNC('quarter', CURRENT_DATE - INTERVAL '3 months') + INTERVA
  SUM(cantidad_vendida) AS total_unidades
FROM
  ventas_trimestre
GROUP BY
  producto_nombre, producto_categoria, producto_subcategoria
ORDER BY
  total_unidades DESC
```

10.6. Conjunto de Datos

El conjunto de datos que hemos seleccionado para implementar nuestro sistema de NL-2-SQL es el conocido conjunto AdventureWorks, disponibilizado como conjunto de pruebas por Microsoft. Este conjunto de datos representa una empresa ficticia de fabricación de bicicletas llamada AdventureWorks Cycles y es todo un clásico dentro del mundo del Business Inteligence como conjunto de pruebas. Adventure Works es una gran empresa multinacional de fabricación que produce y distribuye bicicletas, piezas y accesorios para mercados comerciales en Norteamérica, Europa y Asia. La empresa tiene 500 trabajadores. Además, Adventure Works emplea varios equipos de ventas regionales en su base comercial.

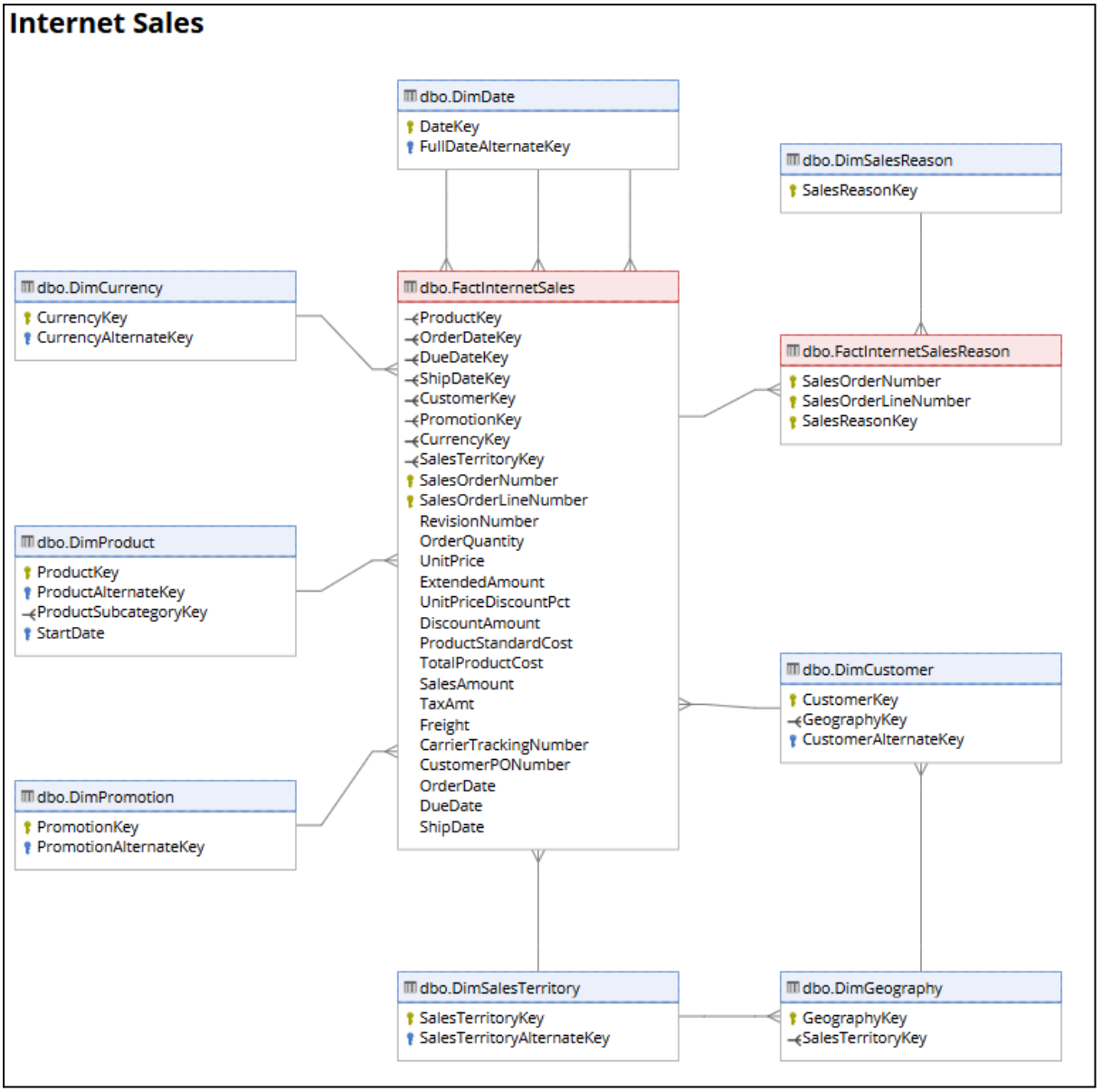
A continuación, en enlace del repositorio de donde se han obtenido los datos:
https://github.com/microsoft/sql-server-samples/tree/master/samples/databases/adventure-works/data-warehouse-install-script

El primer desafío para poder implementar este conjunto de datos es que Microsoft, se trata de que, si bien la empresa lo ofrece bajo licencia MIT, el script de instalación que facilita es para ser utilizado en Microsoft SQL Server, lo que hace que no se pueda ejecutar directamente en nuestro motor de base de datos de Postgres. Para resolver este punto, se ha creado un script que, primero se encarga de limpiar el script, extrayendo de él únicamente las secciones de interés, para luego pasar este extracto a un LLM encargado de "traducir" el mismo del dialecto T-SQL, utilizado por SQL Server, a código ejecutable por Postgres.

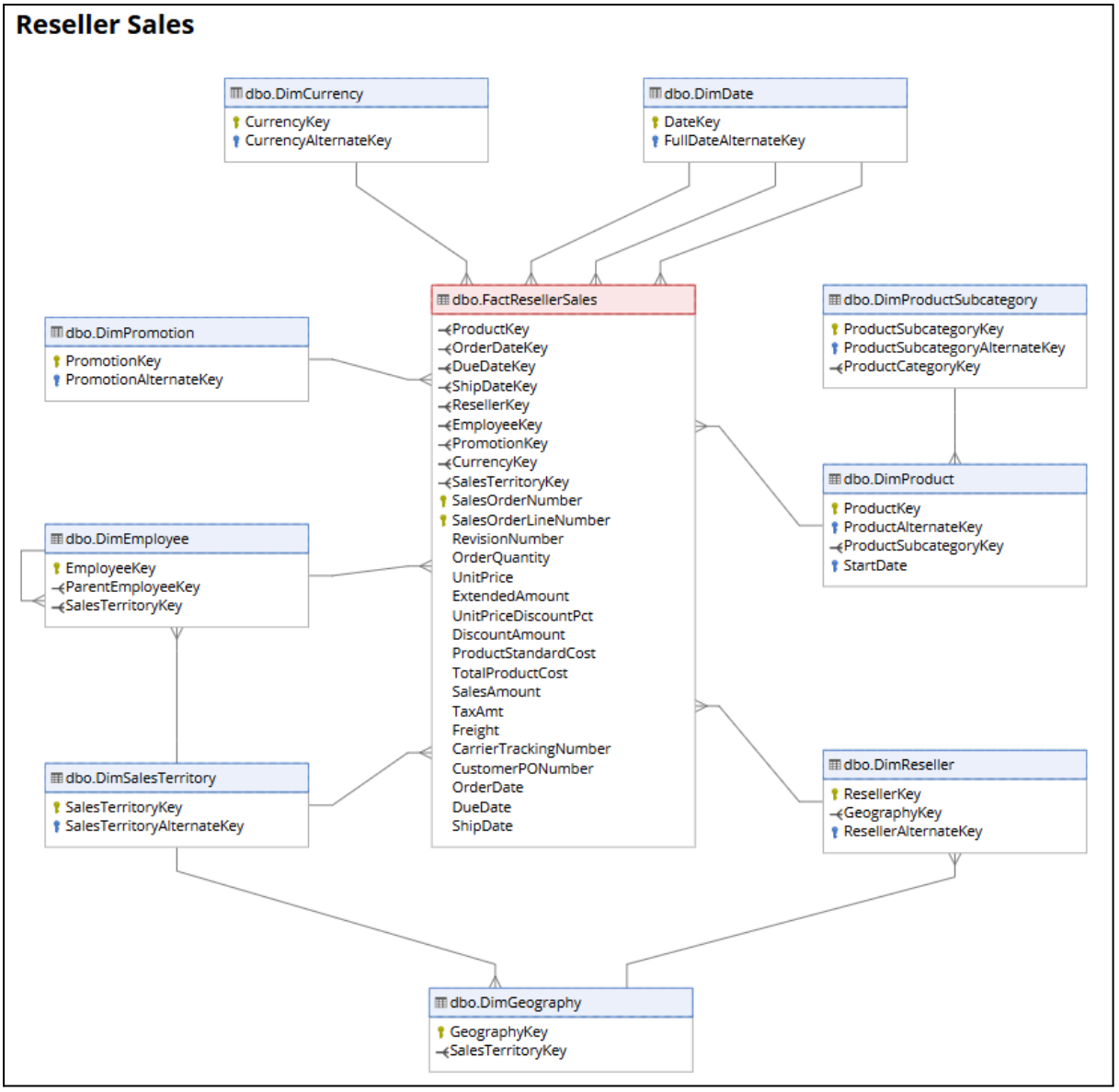
El notebook se puede consultar en el siguiente enlace:
https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/notebooks/translate_tsql_to_postgres.ipynb

El script obtenido tras este proceso, se puede consultar en el siguiente enlace:
https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/data/database/postgres/db_creation/create_adventure_works_postgres.sql

Una vez ejecutado el script anterior, se obtiene el conjunto de datos original, es decir, tal y como lo provee Microsoft, el cual cubre varios escenarios empresariales como fabricación, ventas, compras, gestión de productos, gestión de contactos y recursos humanos. Si bien los datos ya vienen con un esquema de modelado dimensional que contempla todos los aspectos antes nombrados, para nuestro proyecto hemos decidido generar un modelo dimensional reducido, enfocado principalmente en el esquema de ventas, que la base de datos original hace una distinción entre ventas por internet (a clientes directos) y a revendedores. A continuación, se puede ver cómo se conforma el esquema original para las tablas relacionadas a ventas:



Internet Sales - AdventureWorksDW



Reseller Sales - AdventureWorksDW

Sobre las tablas mostradas en los esquemas anteriores, se ejecuta un pipeline de transformación, que nos permite obtener el esquema reducido planteado anteriormente, unificando algunas de las tablas, y haciendo algunas adaptaciones sobre los valores, como por ejemplo, ajustando las fechas para que el rango temporal se ajuste a fechas actuales. Este pipeline está montado en la sección “Esquema de Ventas Reducido (Sales)” del notebook del siguiente enlace: https://github.com/grimbano/ucm-tfm-grupo-4/blob/develop/notebooks/create_reduced_db_schemas.ipynb

Tras aplicar todas las transformaciones comentadas, el esquema obtenido es el siguiente:

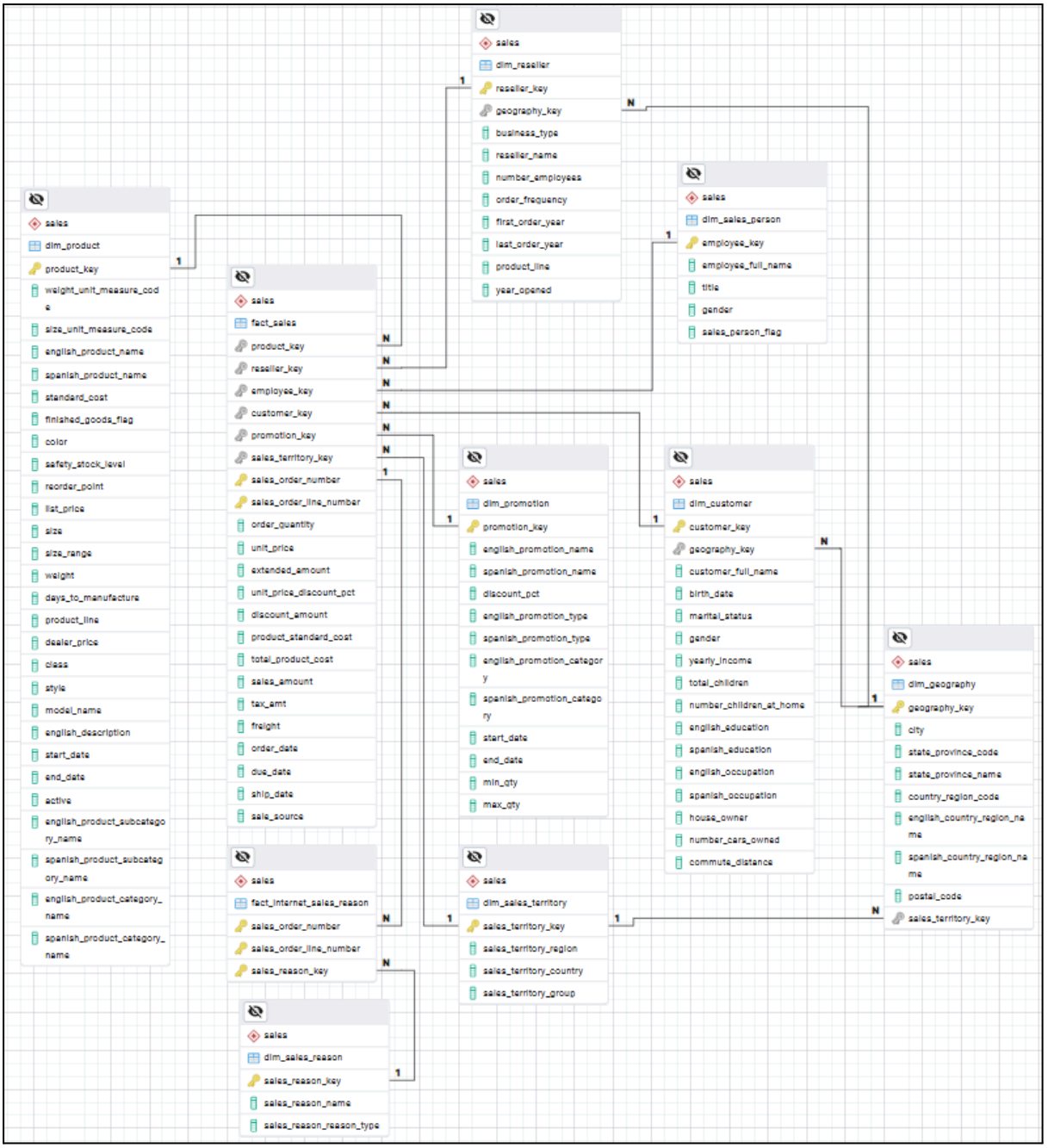

sales
dim_product
product_key
weight_unit_measure_code
size_unit_measure_code
english_product_name
spanish_product_name
standard_cost
finished_goods_flag
color
safety_stock_level
reorder_point
list_price
size
size_range
weight
days_to_manufacture
product_line
dealer_price
class
style
model_name
english_description
start_date
end_date
active
english_product_subcategory_name
spanish_product_subcategory_name
english_product_category_name
spanish_product_category_name
sales
fact_sales
product_key
reseller_key
employee_key
customer_key
promotion_key
sales_territory_key
sales_order_number
sales_order_line_number
order_quantity
unit_price
extended_amount
unit_price_discount_pct
discount_amount
product_standard_cost
total_product_cost
sales_amount
tax_amt
freight
order_date
due_date
ship_date
sale_source
sales
dim_reseller
reseller_key
geography_key
business_type
reseller_name
number_employees
order_frequency
first_order_year
last_order_year
product_line
year_opened
sales
dim_sales_person
employee_key
employee_full_name
title
gender
sales_person_flag
sales
dim_promotion
promotion_key
english_promotion_name
spanish_promotion_name
discount_pct
english_promotion_type
spanish_promotion_type
english_promotion_category
spanish_promotion_category
start_date
end_date
min_qty
max_qty
sales
dim_customer
customer_key
geography_key
customer_full_name
birth_date
marital_status
gender
yearly_income
total_children
number_children_at_home
english_education
spanish_education
english_occupation
spanish_occupation
house_owner
number_cars_owned
commute_distance
sales
dim_geography
geography_key
city
state_province_code
state_province_name
country_region_code
english_country_region_name
spanish_country_region_name
postal_code
sales_territory_key
sales
fact_internet_sales_reason
sales_order_number
sales_order_line_number
sales_reason_key
sales
dim_sales_territory
sales_territory_key
sales_territory_region
sales_territory_country
sales_territory_group
sales
dim_sales_reason
sales_reason_key
sales_reason_name
sales_reason_reason_type

Tabla	Registros	Descripción
sales.dim_customer	18.484	Tabla de dimensión que almacena la información detallada de cada cliente que haya comprado online (por internet), para el segmento minorista (B2C).
sales.dim_geography	655	Tabla de dimensión que almacena información detallada sobre datos geográficos.
sales.dim_product	350	Tabla de dimensión que almacena la información detallada de cada producto que alguna vez haya sido vendido. En esta tabla se han unificado todos los datos de productos, incluyendo sus categorías y subcategorías, que en el esquema original eran tablas independientes.
sales.dim_promotion	16	Tabla de dimensión que almacena la información detallada de las diferentes promociones / descuentos que pueden aplicarse a las ventas.
sales.dim_reseller	701	Tabla de dimensión que almacena la información detallada de cada tienda/distribuidor del segmento mayorista (B2B).
sales.dim_sales_person	17	Tabla de dimensión que almacena la información detallada de cada vendedor.
sales.dim_sales_reason	10	Tabla de dimensión que almacena la información detallada de cada motivo de venta.
sales.dim_sales_territory	11	Tabla de dimensión que almacena la información detallada de cada cada división territorial de ventas.
sales.fact_internet_sales_reason	64.515	Tabla de referencias cruzadas que asigna líneas de órdenes de ventas a códigos de motivo de venta.
sales.fact_sales	121.253	Tabla de hechos que contiene el detalle de las órdenes de ventas que ya han sido entregadas, con una granularidad a nivel de línea, mostrando siempre la última versión de cada pedido. Permite hacer desgloses a nivel de productos, clientes, tiendas/distribuidores, división territorial, promociones aplicadas y vendedores involucrados.